

# Learning Without Forgetting for Vision-Language Models

Da-Wei Zhou<sup>✉</sup>, Yuanhan Zhang, Yan Wang, Jingyi Ning<sup>✉</sup>, Han-Jia Ye<sup>✉</sup>, De-Chuan Zhan<sup>✉</sup>,  
and Ziwei Liu<sup>✉</sup>, *Member, IEEE*

**Abstract**—Class-Incremental Learning (CIL) or continual learning is a desired capability in the real world, which requires a learning system to adapt to new tasks without forgetting former ones. While traditional CIL methods focus on *visual* information to grasp core features, recent advances in Vision-Language Models (VLM) have shown promising capabilities in learning generalizable representations with the aid of *textual* information. However, when continually trained with new classes, VLMs often suffer from catastrophic forgetting of former knowledge. Applying VLMs to CIL poses two major challenges: 1) how to adapt the model without forgetting and 2) how to make full use of the multi-modal information. To this end, we propose PROjectiOn Fusion (PROOF) that enables VLMs to learn without forgetting. To handle the first challenge, we propose training task-specific projections based on the frozen image/text encoders. When facing new tasks, new projections are expanded, and former projections are fixed, alleviating the forgetting of old concepts. For the second challenge, we propose the fusion module to better utilize the cross-modality information. By jointly adjusting visual and textual features, the model can capture better task-specific semantic information that facilitates recognition. Extensive experiments on nine benchmark datasets with various continual learning scenarios and various VLMs validate that PROOF achieves state-of-the-art performance.

**Index Terms**—Class-incremental learning, vision-language model, continual learning, catastrophic forgetting.

Received 4 March 2024; revised 26 December 2024; accepted 8 February 2025. Date of publication 11 February 2025; date of current version 7 May 2025. This work was supported in part by National Science and Technology Major Project under Grant 2022ZD0114805, in part by NSFC under Grant 62476123, Grant 62376118, Grant 62006112, Grant 62250069, in part by the Fundamental Research Funds for the Central Universities under Grant 2024300373 and Grant 14380021, in part by CCF-Tencent Rhino-Bird Open Research Fund under Grant RAGR20240101, in part by the Collaborative Innovation Center of Novel Software Technology and Industrialization, in part by China Scholarship Council, in part by the AI & AI for Science Project of Nanjing University, Ministry of Education, Singapore, under its MOE AcRF Tier 2 under Grant MOET2EP20221-0012, in part by NTU NAP, and in part by Industry Alignment Fund – Industry Collaboration Projects (IAF-ICP) Funding Initiative under Grant RIE2020. Recommended for acceptance by V. Morariu. (*Corresponding authors: Han-Jia Ye; Ziwei Liu.*)

Da-Wei Zhou, Yan Wang, Jingyi Ning, Han-Jia Ye, and De-Chuan Zhan are with the School of Artificial Intelligence, National Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: zhoudw@lamda.nju.edu.cn; wangy@lamda.nju.edu.cn; ningy@nju.edu.cn; yehj@lamda.nju.edu.cn; zhanc@lamda.nju.edu.cn).

Yuanhan Zhang and Ziwei Liu are with the S-Lab, College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: yuanhan002@e.ntu.edu.sg; ziwei.liu@ntu.edu.sg).

Code is available at <https://github.com/zhou-dw-zdw/PROOF>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2025.3540889>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2025.3540889

## I. INTRODUCTION

IN OUR ever-changing world, training data often comes in a stream format with new classes, requiring a learning system to absorb them continually [1], [2], [3], [4], [5]. To address the challenge of learning emerging new classes, Class-Incremental Learning (CIL) has been proposed [6]. However, in CIL, the absence of former classes triggers catastrophic forgetting [7], where learning new concepts overwrites the knowledge of old ones and results in a decline in performance [8]. Numerous efforts have been made [9], [10] to combat catastrophic forgetting in the machine learning field.

With the rapid development of pre-training techniques [11], [12], [13], [14], recent years have witnessed the transition of CIL research from training from scratch [15], [16], [17] to *utilizing pre-trained models* (PTM) [18], [19], [20], [21]. With the help of PTM, e.g., Vision Transformers [22], incremental learners are born with strong transferability to grasp the *visual* features. Facing the domain gap introduced by the incremental classes, they only need to learn a limited number of additional parameters [23], [24], [25] as the *patches* to bridge the distribution gap, which significantly simplifies the challenge of incremental learning.

While pre-trained ViT-based CIL methods focus on learning the *visual* features to recognize new concepts, recent advances in Vision-Language Models (VLM) have demonstrated the potential of *textual* information in building generalized feature representations. A seminal work, i.e., contrastive language-image pre-training [26] (CLIP), maps the visual and textual information in the shared embedding space, enabling robust learning and recognition of concepts from diverse sources. This integration of visual and textual modalities presents a promising avenue for developing continual learning models that can effectively adapt to real-world scenarios.

Extending VLMs to CIL faces two significant challenges. First, sequentially tuning the VLM overwrites the innate generalizability and former concepts, with the former leading to poor performance on future tasks and the latter to catastrophic forgetting. Second, relying solely on textual information for classification neglects the valuable cross-modal features present in the multi-modal inputs. To fully utilize this information, it is necessary to explore methods for cross-modal fusion beyond textual features.

Correspondingly, we aim to turn a VLM into a continual learner that is both *retentive* and *comprehensive* so that VLMs

can be updated in an incremental manner. Retentive refers to the model's ability to maintain its pre-trained capabilities, thereby preserving generalizability and enabling it to perform well on future tasks without forgetting. Comprehensive refers to the model's capacity to integrate and adjust information from multiple modalities. By leveraging these characteristics, we can mitigate catastrophic forgetting and use cross-modal features to build more robust classifiers as data evolves.

In this paper, we propose PROjectiOn Fusion (**PROOF**) to address catastrophic forgetting in VLM. To make the model retentive, we freeze the pre-trained image/text backbones and append linear projections on top of them. The task-specific information is encoded in the corresponding projection layer by mapping the projected features. When facing new tasks, new projections are extended while old ones are frozen, preserving former knowledge. Besides, we aim to fuse the information from different modalities via cross-modal fusion, which allows for the query embedding to be adjusted with context information. Consequently, PROOF efficiently incorporates new classes and meanwhile resists forgetting old ones, achieving state-of-the-art performance on nine benchmark datasets and a non-overlapping TV series classification dataset. We also evaluate PROOF in various continual learning settings, including CIL and continual cross-modal retrieval, to show its effectiveness in various real-world scenarios. Our contributions can be summarized as follows:

- We propose a general framework that enables a pre-trained vision-language model to continually learn new classes without catastrophic forgetting;
- We design a novel projection fusion mechanism to enhance the model's representation ability and a cross-modal fusion module to encode task-specific information. We build the aggregated inference format considering cross-modal matching targets holistically;
- PROOF achieves state-of-the-art performance on nine benchmark datasets and a non-overlapping dataset. Benefiting from its universality, PROOF also shows strong performance on continual cross-modal retrieval tasks against other cutting-edge methods.

## II. RELATED WORK

### A. Vision-Language Model (VLM) Tuning

Recent years have witnessed the prosperity of research in vision-language models, e.g., CLIP [26], ALIGN [27], CoCa [28], Florence [29], BLIP [30], CLIPPO [31], and Flamingo [32]. These models are pre-trained on vast amounts of images and texts, achieving a unified embedding space across different modalities. With great generalizability, they can be applied for downstream tasks in a zero-shot manner. However, a domain gap still exists between the pre-trained and downstream datasets, requiring further tuning for better performance. To fill this gap, many methods are proposed to tune a pre-trained VLM for downstream tasks. CoOp [33] applies prompt learning [34] into VLM tuning with learnable prompt tokens for the textual branch, where a set of learnable

prompts are utilized to replace the template texts. CoCoOp [35] further encodes the instance-specific visual features into the learnable prompts. CLIP-Adapter [36] appends linear adapters after the visual and textual encoders to align the embeddings in the adapted space. ProDA [37] introduces prompt distribution learning into the prompt learning process, and TaskRes [38] directly learns a task-wise residual feature to bridge the domain gap. Tip-Adapter [39] caches visual prototypes and combines them with textual encoded information to construct a cross-modal inference paradigm. Moreover, PLOT [40] optimizes cross-modal matching by aligning multiple local visual features with textual prompts via optimal transport [41]. [42] proposes a prompt learning technique to tackle the missing modality with a pre-trained multi-modal transformer. Recent works also utilize ChatGPT as auxiliary knowledge to enhance the cross-modal mapping process [43], [44]. However, these works only focus on adapting VLMs to downstream tasks while overlooking the *catastrophic forgetting* of previous ones. When deploying VLMs into a sequence of downstream tasks, a desired algorithm should handle all tasks without forgetting.

### B. Class-Incremental Learning (CIL)

Class-Incremental Learning aims to learn from evolutive data and absorb new knowledge without forgetting [9], [10], [21], [45], [46], which can be divided into several groups. Replay-based methods [47], [48], [49], [50], [51] save and replay former instances (i.e., exemplars) to recover old knowledge when learning new ones. Apart from directly replaying raw images, there are also works considering replaying features [52], low-resolution images [53], and logits [54]. Moreover, generative models are also widely applied to model the distribution of previous tasks for replay, e.g., GAN [55], [56], VAE [57], diffusion model [58], [59]. The second group utilizes knowledge distillation [60] to build the mapping between models as regularization term [6], [8], [61]. iCaRL [6] and LwF [8] explore the logit-wise distillation to align the predictions between old and new models to resist forgetting. Besides, [62], [63], [64] build the feature-wise mapping, and [65], [66], [67] regularize the group-wise information via relational distillation. The third group builds parameter-wise regularization terms to force important parameters not to drift away [68], [69], [70], [71]. The fourth group locates and rectifies the inductive bias of CIL models for unbiased predictions [15], [17], [72], [73]. For example, BiC [15] finds the predicted logits of the latest task are much higher than previous ones and designs a bias correction layer to calibrate the prediction. IL2M [74] calibrates the logits via re-scaling the task-wise predictions, while WA [17] directly normalizes the fully-connected layers for an unbiased prediction. The last group designs dynamic networks [75], [76], [77] by expanding the network structure as data evolves. The network expansion techniques are further divided into neuron-wise [78], [79], backbone-wise [75], [76], [77], [80], and token-wise [81]. Besides, OSN [82] contains shared knowledge induced network partition and sharpness-aware orthogonal sparse network learning, aiming to enhance the plasticity and capacity.

### C. CIL With VLM

The aforementioned CIL algorithms aim to train an incremental model from scratch, while it would be easier to start with a pre-trained model [83]. The integration of pre-trained Vision Transformer [22] into CIL has attracted the attention of the community, and most methods [18], [19], [20] employ parameter-efficient tuning techniques to learn without forgetting. L2P [19] introduces the prompt pool and prompt search mechanism in CIL. It freezes the pre-trained weights, optimizes a set of visual prompts, and searches for the most similar prompts for instance-specific embeddings. DualPrompt [18] further explores the prompt depth and shared prompt for all tasks. CODA-Prompt [20] replaces the key-value search mechanism and designs an attention-based prompt calculation strategy. NSP<sup>2</sup> [84] aims to learn each task by tuning the prompts in the direction orthogonal to the subspace spanned by previous tasks' features, so as to ensure no interference on tasks that have been learned to overcome catastrophic forgetting. Following works also consider generating prompts via a meta-network [85], [86] or aggregating predictions via a set of adjusted models [87], [88]. However, these works are designed for pre-trained ViT and lack the potential to be compatible with vision-language models with multiple modality information. S-Prompts [89] explores CLIP in *domain*-incremental learning, but the application of VLM in CIL remains relatively unexplored. WiSE-FT [90] utilizes weight ensemble for robust finetuning, while it cannot be extended to multiple tasks. This paper aims to address this research gap by presenting a comprehensive solution for tuning vision-language models without suffering from forgetting.

## III. PRELIMINARIES

In this section, we introduce the background information about class-incremental learning and vision language models. We also discuss the naïve solutions for tuning VLM in CIL.

### A. Class-Incremental Learning

Given a data stream with emerging new classes, class-incremental learning aims to continually incorporate the knowledge and build a unified classifier [45]. We denote the sequence of  $B$  training sets without overlapping classes as  $\{\mathcal{D}^1, \mathcal{D}^2, \dots, \mathcal{D}^B\}$ , where  $\mathcal{D}^b = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n_b}$  is the  $b$ -th training set with  $n_b$  instances. A training instance  $\mathbf{x}_i \in \mathbb{R}^D$  belongs to class  $y_i \in Y_b$ .  $Y_b$  is the label space of task  $b$ , and  $Y_b \cap Y_{b'} = \emptyset$  for  $b \neq b'$ . Following the typical CIL setting [6], [15], [62], a fixed number of *exemplars* from the former classes are selected as the exemplar set  $\mathcal{E}$ . During the  $b$ -th incremental stage, we can only access data from  $\mathcal{D}^b$  and  $\mathcal{E}$  for model training. The target is to build a unified classifier for all seen classes  $\mathcal{Y}_b = Y_1 \cup \dots \cup Y_b$  continually. In other words, we aim to find a model  $f(\mathbf{x}) : X \rightarrow \mathcal{Y}_b$  that minimizes the expected risk:

$$f^* = \operatorname{argmin}_{f \in \mathcal{H}} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_1^1 \cup \dots \mathcal{D}_t^t} \mathbb{I}(y \neq f(\mathbf{x})), \quad (1)$$

where  $\mathcal{H}$  denotes the hypothesis space and  $\mathbb{I}(\cdot)$  is the indicator function.  $\mathcal{D}_t^b$  denotes the data distribution of task  $b$ . Following [18], [19], [89], we assume that a pre-trained vision-language

model is available as the initialization for  $f(\mathbf{x})$ , which will be introduced in Section III-B.

### B. Vision-Language Model

This paper mainly focuses on contrastive language-image pre-training (CLIP) [26] as the VLM, while the proposed method is also compatible with other VLMs in Section V-D. During pre-training, CLIP jointly learns an image encoder  $g_i(\cdot) : \mathbb{R}^D \rightarrow \mathbb{R}^d$  and a text encoder  $g_t(\cdot) : \mathbb{R}^{D_t} \rightarrow \mathbb{R}^d$  in a contrastive manner, where  $D/D_t$  are input dimensions of image/text, and  $d$  is the embedding dimension. CLIP projects a batch of image-text pairs into a shared embedding space. It maximizes the cosine similarity of paired inputs and minimizes it for unmatched ones. Benefiting from the massive training data, CLIP can synthesize a *zero-shot classifier* that generalizes to unseen classes. The output of CLIP is formulated as follows:

$$p(y_i | \mathbf{x}) = \frac{\exp(\cos(\mathbf{z}, \mathbf{w}_i) / \tau)}{\sum_{j=1}^{|\mathcal{Y}_b|} \exp(\cos(\mathbf{z}, \mathbf{w}_j) / \tau)}, \quad (2)$$

where  $\cos(\cdot, \cdot)$  denotes cosine similarity,  $\tau$  is learnable temperature parameter,  $\mathbf{z} = g_i(\mathbf{x})$  is the image embedding. Correspondingly,  $\mathbf{w}_i$  is the text embedding of class  $y_i$  obtained by feeding templated texts, e.g., “a photo of a [CLASS]” into the text encoder. We denote the templated text of class  $i$  as  $\mathbf{t}_i$ . Equation (2) aims to find the most similar text  $\mathbf{t}_i$  that maximizes the cosine similarity with the query image.

### C. Overcome Forgetting in Class-Incremental Learning

Class-incremental learning, as a long-standing problem, has garnered significant attention from the research community. In this section, we introduce two typical solutions for adapting pre-trained models with new classes and discuss their limitations.

*Vision-Based Learning:* Traditional CIL methods primarily rely on the *image encoder* to capture the patterns of new classes. One such method, L2P [19], leverages visual prompt tuning [23] to enable incremental updates of a pre-trained Vision Transformer [22]. By keeping the image encoder frozen, L2P trains a learnable prompt pool **Pool** and combines it with patch embeddings to obtain instance-specific embeddings. The optimization target can be formulated as:

$$\mathcal{L} = \ell(h(\bar{g}_i(\mathbf{x}_i, \mathbf{Pool})), y_i) + \mathcal{L}_{reg}, \quad (3)$$

where  $h(\cdot)$  is the classification head,  $\bar{g}_i$  is the frozen image encoder,  $\mathcal{L}_{reg}$  is the regularization loss for prompt selection. By freezing the encoder, Equation (3) grasps the new pattern with limited forgetting.

*CLIP Tuning:* The issue of tuning VLM without forgetting in CIL remains unaddressed, as previous works have solely focused on transferring CLIP to downstream tasks without considering the performance of former tasks. For instance, CoOp [33] converts text template into a learnable prompt, i.e.,  $\mathbf{t}_i = [\mathbf{V}]_1[\mathbf{V}]_2 \dots [\mathbf{V}]_M[\mathbf{CLASS}]_i$  and changes Equation (2) into:

$$p(y_i | \mathbf{x}) = \frac{\exp(\cos(\mathbf{z}, g_t(\mathbf{t}_i)) / \tau)}{\sum_{j=1}^{|\mathcal{Y}_b|} \exp(\cos(\mathbf{z}, g_t(\mathbf{t}_j)) / \tau)}. \quad (4)$$



With the help of the learned prompt, Equation (4) enables the model to be transferred to the downstream task. However, since the prompt template is shared for all tasks, sequentially tuning CoOp will suffer catastrophic forgetting of former concepts.

*Discussions:* Current methods focus on different aspects of CIL. Vision-based methods (e.g., Equation (3)) address the issue of forgetting but neglect the valuable semantic information conveyed in texts. Conversely, CLIP's pre-trained text encoder captures class-wise relationships that can enhance model learning. Meanwhile, transfer learning methods (e.g., Equation (4)) effectively leverage the cross-modal information while sequentially tuning them suffers the catastrophic forgetting of former concepts. Is it possible to combine the cross-modal information while resisting catastrophic forgetting?

#### IV. PROOF: PROJECTION FUSION FOR VLM

Observing the limitations of typical vision-based methods in utilizing textual information and the forgetting phenomenon in CLIP tuning, we aim to leverage cross-modality knowledge in CLIP while effectively mitigating forgetting. To this end, we must make the model *retentive* and *comprehensive*. Retentive represents the ability to adapt to downstream tasks without forgetting, and we propose applying projections to map the pre-trained features into the projected feature space. Our unique training strategy ensures the preservation of former knowledge by freezing old projections and expanding new ones for new tasks. The comprehensive aspect involves co-adapting and utilizing cross-modal information to enhance unified predictions. The query instance's embedding is influenced by both visual and textual information, allowing for instance-specific adaptation and enabling comprehensive predictions. In the following sections, we introduce the learning paradigm and the co-adaptation process. Lastly, we provide detailed guidelines for training and inference.

##### A. Expandable Feature Projection

CLIP is known for its strong zero-shot performance, i.e., Equation (2) obtains competitive results even without explicit training on the specific tasks. However, given the domain gap between pre-trained and downstream tasks, an *adaptation* process is still necessary to capture the characteristics of the latter. Specifically, we introduce a *linear layer* (denoted as “**projection**”), which is appended after the frozen image and text embeddings to facilitate the matching of pair-wise projected features. Denoting the projection of image/text as  $P_i(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^d$  and  $P_t(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ , Equation (2) is transformed into:

$$p(y_i | \mathbf{x}) = \frac{\exp(\cos(P_i(\mathbf{z}), P_t(\mathbf{w}_i)) / \tau)}{\underbrace{\sum_{j=1}^{|\mathcal{Y}_b|} \exp(\cos(P_i(\mathbf{z}), P_t(\mathbf{w}_j)) / \tau)}_{\text{Projected Matching}}}. \quad (5)$$

We denote the classification based on Equation (5) as  $f_{\text{PM}}(\mathbf{x})$ . By freezing the image and text encoders, the downstream features in the projected space are aligned, allowing the model to encode the relevant downstream information into projection

layers. Since the pre-trained model outputs generalizable features, the projection layer learns to *recombine* features in a data-driven manner. For instance, in a task involving ‘birds,’ the projection would assign a higher weight to features like ‘beaks’ and ‘wings.’ This adaptation enables the projected features to better discern and recognize downstream tasks.

*Expandable Projections:* However, sequentially training a *single* projection layer still leads to forgetting of former tasks, resulting in confusion when combining old and new concepts. To this end, we expand *task-specific* projections for each new task. Specifically, we append a newly initialized projection layer  $P_i^b, P_t^b$  when a new task  $\mathcal{D}^b$  arrives. This results in a set of projections:  $\{P_i^1, P_i^2, \dots, P_i^b\}$ ,  $\{P_t^1, P_t^2, \dots, P_t^b\}$ , and we adopt the *aggregation* as the output:

$$P_i(\mathbf{z}) = \sum_{m=1}^b P_i^m(\mathbf{z}), \quad P_t(\mathbf{w}) = \sum_{n=1}^b P_t^n(\mathbf{w}). \quad (6)$$

In Equation (6), projected features from different stages are mapped and aggregated to capture the different emphases of former and latter tasks. For example, former tasks might emphasize ‘beak’ features for bird recognition, while later tasks may focus on ‘beard’ features to differentiate cats. The aggregation of different projections produces a comprehensive representation of the query instance. By substituting Equation (6) into Equation (5), the model aligns the unified features in the joint space. Since Equation (6) only aggregates the projected features as the final representation, it has no restriction on the number of classes per task.

*How to resist forgetting of former projections?* To overcome forgetting old concepts, we freeze the projections of former tasks when learning new ones, i.e.,  $\{\bar{P}_i^1, \bar{P}_i^2, \dots, \bar{P}_i^b\}$  (same for  $P_t$ ). It allows the newly initialized projection to learn the *residual* information of new tasks, incorporating new concepts while preserving the knowledge of former ones. During the learning process of task  $b$ , we optimize the cross-entropy loss to encode the task-specific information into the current projections.

*Effect of projections:* The illustration of projections is shown in Fig. 1 (left). PROOF learns projections based on the pre-trained encoders, which fits new patterns and maintains the generalizability of the pre-trained model. The parameter number of each projection layer is  $d \times d$ , which is negligible for the pre-trained model. These projections can be further merged during inference to alleviate the storage budget, as discussed in Section V-C6. Furthermore, the model learns new projections for new tasks, and task-specific projections fit new concepts easily. Since we only optimize the current projections and freeze old ones, the former knowledge is preserved, and forgetting is alleviated.

##### B. Contextualizing Projections With Projection Fusion

In Equation (5), the projected visual and textual features are directly matched in the joint space. However, it would be beneficial to further *refine* these features to capture the *contextual relationship* between images and texts. For instance, when the query instance is a ‘panda,’ it is desirable to adjust the visual and textual features in a *coherent* manner to highlight discriminative attributes such as *black eyes and ears*. Similarly, when the

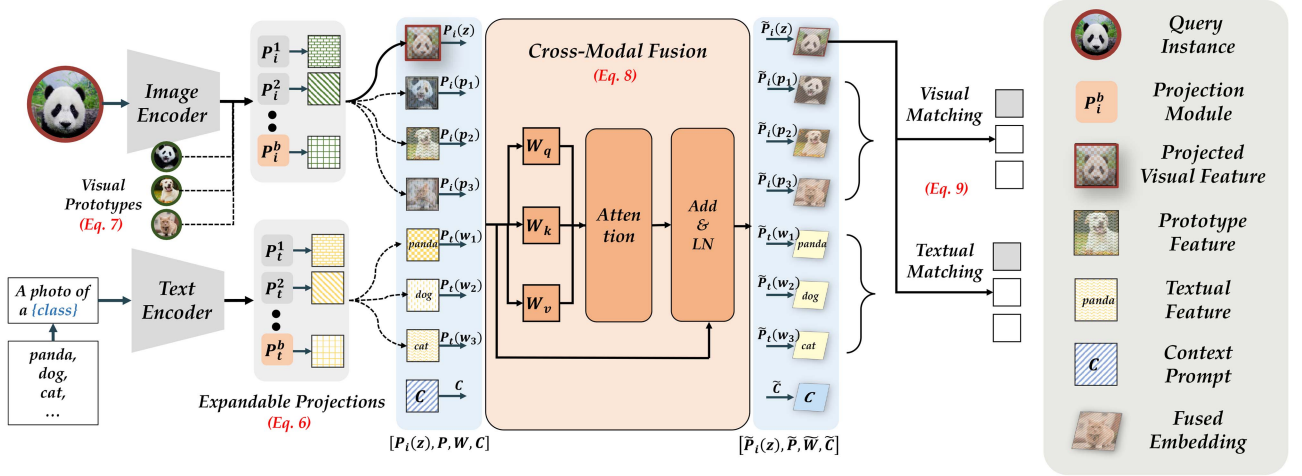


Fig. 1. Illustration of PROOF. The model learns expandable projections and aggregates them to get the aggregated features. The query instance, prototype features, textual features, and context prompts are fed into the cross-modal fusion module. The fusion process utilizes self-attention to co-adapt the input set, which outputs the adapted features. The adapted query embedding is separately matched among the visual prototypes and textual features to get the final prediction. Red parts are trainable while blue ones are frozen.

query instance is a ‘cat,’ features like beards and tails should be emphasized. This adjustment process involves jointly adapting the query embedding and the context (e.g., textual information) to obtain a *contextualized* embedding. Correspondingly, we propose a *set-to-set* function that contextualizes and fuses the query embeddings and contextual information.

Specifically, we denote the adaptation function as  $\mathcal{T}(\cdot)$ . It receives the query instance and context information as bags, i.e.,  $[P_i(\mathbf{z}), \text{Context}]$ , and outputs the set of adjusted embeddings while being permutation-invariant:  $\mathcal{T}([P_i(\mathbf{z}), \text{Context}]) = [\tilde{P}_i(\mathbf{z}), \text{Context}]$ .  $\mathcal{T}(\cdot)$  encodes the set information and performs adaptation on each component. In the following, we describe the construction of the context information **Context** and provide details on the implementation of the set-to-set function.

*How to define the context?* In Equation(5), the mapping is established between the query instance and the textual information (i.e., classifiers). The classifiers represent the typical textual description of the corresponding class, i.e., the common feature. Hence, a naïve idea is to utilize textual features as the context, i.e.,  $\mathbf{W} = [P_t(\mathbf{w}_1), P_t(\mathbf{w}_2), \dots, P_t(\mathbf{w}_{|\mathcal{Y}_b|})] \in \mathbb{R}^{|\mathcal{Y}_b| \times d}$  (the concatenation of all textual classifiers). However, recent works find an inherent domain gap [91] between the visual and textual embeddings in VLM. The gap leads to visual and textual embeddings residing in two separate clusters in the embedding space, which hinders effective pair-wise mapping. Correspondingly, we leverage visual prototype features [92] as a useful tool for capturing the common characteristics of each class. We define the *visual prototype* of class  $k$  as:

$$\mathbf{p}_k = \frac{1}{N} \sum_{j=1}^{|\mathcal{D}^b|} \mathbb{I}(y_j = k) g_i(\mathbf{x}_j), \quad (7)$$

where  $N = \sum_{j=1}^{|\mathcal{D}^b|} \mathbb{I}(y_j = k)$ . They are calculated via forward pass at the beginning of each incremental stage and stay fixed

in subsequent tasks. Visual prototypes are *representative* features of the corresponding class, which can serve as the *visual context* to adjust the embeddings. Hence, we augment the context with projected visual information, i.e.,  $[\mathbf{P}, \mathbf{W}]$ , where  $\mathbf{P} = [P_i(\mathbf{p}_1), P_i(\mathbf{p}_2), \dots, P_i(\mathbf{p}_{|\mathcal{Y}_b|})] \in \mathbb{R}^{|\mathcal{Y}_b| \times d}$  is the concatenation of all visual prototypes. Combining prototypes from multiple modalities helps the model adapt and fuse information in a cross-modal manner, which goes beyond simple visual-textual matching.

*Learning Context Prompts:* In addition to visual prototypes and textual classifiers, we also introduce a set of learnable *context prompts*  $\{\mathbf{c}^1, \dots, \mathbf{c}^b\}$ ,  $\mathbf{c}^i \in \mathbb{R}^{c \times d}$  to be optimized as data evolves.  $c$  denotes the length of each prompt. Similar to projection layers, we make the context prompts *expandable* to catch the new characteristics of new tasks. We initialize a new context prompt while learning a new task and freeze others  $\{\bar{\mathbf{c}}^1, \bar{\mathbf{c}}^2, \dots, \bar{\mathbf{c}}^b\}$ . The context prompts serve as *adaptable* context information, enhancing the co-adaption. Hence, the context information is formulated as **Context** =  $[\mathbf{P}, \mathbf{W}, \mathbf{C}]$ , where  $\mathbf{C}$  is the aggregation of all context prompts.

*Implementing  $\mathcal{T}$  with Self-Attention:* In our implementation, we use the self-attention (SA) mechanism [93], [94] as the cross-modal fusion function  $\mathcal{T}$ . Being permutation invariant, SA is good at outputting adapted embeddings even with long dependencies, which naturally suits the characteristics of the adaptation function. Specifically, SA keeps the triplets (query  $\mathcal{Q}$ , key  $\mathcal{K}$ , and value  $\mathcal{V}$ ). The inputs are projected into the same space, i.e.,  $\mathbf{K} = W_K^\top [\mathbf{k}_k; \forall \mathbf{k}_k \in \mathcal{K}] \in \mathbb{R}^{d \times |\mathcal{K}|}$ . Similar projections are made for  $\mathcal{Q}$  and  $\mathcal{V}$ . The query  $\mathbf{x}_q \in \mathcal{Q}$  is matched against a list of keys  $\mathbf{K}$  where each key has a value  $\mathbf{V}$ . The output is the sum of all the values weighted by the proximity of the key to the query point:

$$\tilde{P}_i(\mathbf{z}) = P_i(\mathbf{z}) + \sum_k \alpha_{qk} V_{:,k}, \quad (8)$$

**Algorithm 1:** Training PROOF for CIL.

---

**Input:** Training dataset:  $\mathcal{D}^b$ ; Exemplar set:  $\mathcal{E}$ ; Current model:  $f(\cdot)$ ;  
**Output:** Updated model;  
 1: Extract prototypes  $\mathbf{p}$  for each new class in  $\mathcal{D}^b$ ;  
 2: Freeze current projections and context prompts;  
 3: Initialize new projections  $P_i^b, P_t^b$ ;  
      $\triangleright$  **Expand projections**  
 4: Initialize new context prompt  $\mathbf{c}^b$ ;  
 5: **for**  $(\mathbf{x}, y) \in \mathcal{D}^b \cup \mathcal{E}$  **do**       $\triangleright$  **Incremental learning**  
 6:   Calculate the visual embedding  $\mathbf{z} = g_i(\mathbf{x})$ ;  
 7:   Calculate projected visual/textual embeddings via (6);  
 8:   Calculate  $f_{\text{PM}}(\mathbf{x})$  via (5);       $\triangleright$  **Projected matching**  
 9:   Cross-modal fusion via (8);       $\triangleright$  **Cross-modal fusion**  
 10:   Calculate visual and textual matching via (9);  
 11:   Calculate the loss via (10); update the model;  
**return** the updated model;

---

where  $\alpha_{qk} \propto \exp(\frac{P_i(\mathbf{z})^\top W_Q \cdot K}{\sqrt{d}})$ ,  $V_{:,k}$  is the  $k$ -th column of  $V$ . The adaptation process is the same for other components in **Context**. Specifically, we have  $\mathcal{Q} = \mathcal{K} = \mathcal{V} = [P_i(\mathbf{z}), \text{Context}] = [P_i(\mathbf{z}), \mathbf{P}, \mathbf{W}, \mathbf{C}]$ . The adapted features are then denoted as  $[\tilde{P}_i(\mathbf{z}), \tilde{\mathbf{P}}, \tilde{\mathbf{W}}, \tilde{\mathbf{C}}]$  to reflect the context information.

*Effect of Cross-Modal Fusion:* The illustration of the projection fusion is shown in Fig. 1 (right). We utilize the visual and textual information of seen classes as context information to help adjust the *instance-specific* embeddings. The fusion model is trained incrementally to adjust embeddings to reflect the context information as data evolves. With the contextualized embeddings, we can conduct the *visual matching* and *textual matching*:

$$f_{\text{VM \& TM}}(\mathbf{x})_{y_i} = \underbrace{\frac{\exp\left(\cos\left(\tilde{P}_i(\mathbf{z}), \tilde{P}_i(\mathbf{p}_i)\right) / \tau\right)}{\sum_{j=1}^{|\mathcal{Y}_b|} \exp\left(\cos\left(\tilde{P}_i(\mathbf{z}), \tilde{P}_i(\mathbf{p}_j)\right) / \tau\right)}}_{\text{Visual Matching}} + \underbrace{\frac{\exp\left(\cos\left(\tilde{P}_i(\mathbf{z}), \tilde{P}_t(\mathbf{w}_i)\right) / \tau\right)}{\sum_{j=1}^{|\mathcal{Y}_b|} \exp\left(\cos\left(\tilde{P}_i(\mathbf{z}), \tilde{P}_t(\mathbf{w}_j)\right) / \tau\right)}}_{\text{Textual Matching}}. \quad (9)$$

In Equation (9), the model assigns logits to the query instance by the similarity to the *adapted* visual and textual prototypes. The incorporation of cross-modal matching enhances the prediction performance. Note that the context prompt  $\mathbf{C}$  only encodes the task-specific information into the self-attention process, i.e., it does not serve as the matching target in Equation (9).

### C. Summary of PROOF

In PROOF, we first enable learning new concepts via projected mapping. Then, to accommodate new concepts without interference from previous ones, we initialize new projections for

each new task and freeze the former ones. Besides, we utilize self-attention to adjust the embeddings of the query instance and the context information to promote cross-modal fusion. Fig. 1 illustrates three matching targets, i.e., projected matching (Equation (5)), visual/textual matching (Equation (9)). We denote these models as  $f_{\text{PM}}(\mathbf{x})$ ,  $f_{\text{VM}}(\mathbf{x})$ ,  $f_{\text{TM}}(\mathbf{x})$ , respectively. During training, we optimize the cross-entropy loss:

$$\min_{\{P_i^b, P_t^b, \tau, \mathbf{c}^b\}} \ell(f_{\text{PM}}(\mathbf{x}), y) + \ell(f_{\text{VM}}(\mathbf{x}), y) + \ell(f_{\text{TM}}(\mathbf{x}), y), \quad (10)$$

where  $(\mathbf{x}, y) \in \mathcal{D}^b \cup \mathcal{E}$ . In Equation (10), all pre-trained weights are frozen, and we only optimize these *additional* parameters. For inference, we aggregate the three logits, i.e.,  $f(\mathbf{x}) = f_{\text{PM}}(\mathbf{x}) + f_{\text{VM}}(\mathbf{x}) + f_{\text{TM}}(\mathbf{x})$ .

*Pseudo Code:* We give the pseudo-code of PROOF to illustrate the training process in Algorithm 1. In each incremental stage, we are provided with the training dataset  $\mathcal{D}^b$  and the exemplar set  $\mathcal{E}$  to update the current model  $f(\cdot)$ . Before training, we initially extract visual prototypes for the new classes. These prototypes are calculated using the frozen visual embedding  $g_i(\cdot)$ , ensuring their stability throughout model updates. Subsequently, we freeze the former projections and context prompts while initializing new projections and context prompts specifically for the new incremental task (Line 2 to Line 4). These steps represent the model expansion process, which is followed by the subsequent learning process.

During the learning process, we concatenate the training instances from the current dataset and the exemplar set, initiating a for-loop. For each instance-label pair, we calculate the projected visual and textual embeddings (Line 6 to Line 7). Subsequently, we compute the projected matching loss (Line 8) to encode task-specific information into the current projection layers. Based on the projected features, we derive context information and perform cross-modal fusion (Line 9 to Line 10). Consequently, we obtain three logits for model updating and utilize the cross-entropy loss to update these modules (Line 11). The updated model is then returned as the output of the training process.

## V. EXPERIMENT

In this section, we compare PROOF to state-of-the-art methods on benchmark datasets to investigate the capability of overcoming forgetting. Besides, we conduct ablations to analyze the effect of each component in the model. We also extend PROOF to other VLMs and continual learning scenarios, experiment with a non-overlapping dataset, and address the zero-shot performance degradation phenomena.

### A. Experimental Setup

*Dataset:* Following the benchmark CIL settings [6], [18], [19], [73], [96], we evaluate the performance on **CI-FAR100** [97], **CUB200** [98], **ObjectNet** [99], and **ImageNet-R** [100]. We also follow the benchmark in VLM tuning [33], and formulate **FGVCAircraft** [101], **StanfordCars** [102], **Food101** [103], **SUN397** [104] and **UCF101** [105] into CIL setting. Specifically, we sample (a subset of) 100 classes



from CIFAR100, Aircraft, Cars, Food, UCF, 200 classes from CUB200, ObjectNet, ImageNet-R, and 300 classes from SUN to ease the data split. Following [6], the training class order is shuffled with random seed 1993. The dataset splits are denoted as **Base- $x$** , **Inc- $y$** , where  $x$  represents the number of classes in the first stage, and  $y$  represents the number of new classes in each subsequent task.  $x = 0$  means each task contains  $y$  classes.

*Comparison methods:* We first compare to SOTA CIL methods iCaRL [6], MEMO [77], SimpleCIL [96], L2P [19], DualPrompt [18]. Denote the baseline of sequential finetuning as Finetune; we combine it with different tuning techniques, e.g., LiT [95], PLOT [40], and CoOp [33]. We also report the zero-shot performance of CLIP as ZS-CLIP by matching the query instance to the template (Equation (2)). Besides, we report the upper bound [15] performance by joint training all tasks, denoted as Oracle. All methods are based on the **same pre-trained CLIP for fair comparison**.

*Implementation details:* We deploy all methods with PyTorch [106] on Tesla V100. We use the *same* network backbone, i.e., CLIP with ViT-B/16 for all compared methods for *fair comparison*. We experiment with two commonly used pre-trained CLIP weights, i.e., OpenAI [26] and OpenCLIP LAION-400M [107]. The model is trained with a batch size of 64 for 5 epochs, and we use SGD with momentum for optimization. The learning rate starts from 0.001 and decays with cosine annealing. Following [6], we use the herding [108] algorithm to select 20 exemplars per class for rehearsal. The context prompt length is set to 3, and the head of self-attention is set to 1. The template for classification in CLIP is kept the same as [109].

*Evaluation Metrics:* Denote the accuracy after the  $b$ -th stage as  $\mathcal{A}_b$ , we follow [6] to use  $\mathcal{A}_B$  (last stage performance) and  $\bar{\mathcal{A}} = \frac{1}{B} \sum_{b=1}^B \mathcal{A}_b$  (average performance) for evaluation.

## B. Benchmark Comparison

We report the results on nine benchmark datasets using CLIP with ViT-B/16 (OpenCLIP LAION-400M) in Table I and Figs. 2 and 3. These splits include the scenarios with large and small base classes. Notably, PROOF consistently achieves the best performance among all the methods compared. Sequential finetuning of the model with contrastive loss leads to significant forgetting, irrespective of the tuning techniques employed (e.g., LiT and CoOp). Since SimpleCIL and ZS-CLIP do not finetune the model parameters, they achieve competitive results by transferring the knowledge from the pre-training stage into the downstream tasks. However, most methods achieve better performance than ZS-CLIP, indicating the importance of incremental learning on downstream tasks. It must be noted that the performance of L2P, DualPrompt, and CODA-Prompt are reproduced with CLIP’s visual branch, which results in a different performance from the original papers. Specifically, we can draw three key conclusions from these results.

- The first stage performance of PROOF surpasses that of the typical prompt learning method, CoOp, thus validating the effectiveness of learning projections for downstream tasks.

- The performance curve of PROOF consistently ranks at the top across all methods, demonstrating its capability to resist forgetting.
- Compared to vision-only methods (i.e., L2P, DualPrompt, CODA-Prompt, DAP), PROOF exhibits substantial improvement, indicating textual and visual information can be co-adapted to facilitate incremental learning.

## C. Ablation Study

1) *Different Backbone Weights:* The comparison in Section V-B is based on LAION-400M pre-trained CLIP. As another popular pre-trained weight, we also explore the performance of the weights provided by OpenAI. As depicted in the figure, PROOF still performs the best on all datasets among all compared methods.

2) *Compositional Components:* We experiment on CIFAR100 B0 Inc10 to investigate the importance of each part in PROOF. Specifically, we compare the performance of PROOF and its sub-modules, i.e., projections and cross-modal fusion. The results, shown in Fig. 4(b), indicate that training expandable projections or the fusion module individually can both enhance the performance of vanilla CLIP. This suggests that the expandable task representation and cross-modal information can help the learning process. Furthermore, when combining them together, we find ‘Projection & Fusion’ further show better performance than any of them, verifying that they can work together by fusing the expandable representations. Lastly, when incorporating the context prompts, the model shows the best performance among all variations, verifying the effectiveness of expandable task-specific prompts in incremental learning. Ablations verify the importance of each component in PROOF.

3) *Number of Context Prompts:* Fig. 4(b) verifies the strong performance of context prompts, and we explore the appropriate length  $c$  of the context prompt on CIFAR100 B0 Inc10. By varying  $c$  among  $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 30, 50, 100\}$ , we report the average performance and last performance of PROOF in Fig. 4(c). It must be noted that the prompt length  $c$  is different from the task number  $B$ , i.e., we only change the scale of each context prompt but maintain one context prompt per incremental task. Correspondingly, we find the performance of PROOF is robust with the change of the prompt length, indicating that context prompt only requires a small scale to encode task-specific information. Hence, we set  $c = 3$  as the default length.

4) *Variation of Context Information:* In this section, we conduct ablations to demonstrate the effectiveness of constructing Context with  $[\mathbf{P}, \mathbf{W}, \mathbf{C}]$ . Specifically, we perform experiments on CIFAR100 B0 Inc10 and change the context construction to  $\mathbf{P}$  (visual prototypes only),  $\mathbf{W}$  (textual prototypes only),  $[\mathbf{P}, \mathbf{W}]$  (visual and textual prototypes), and  $[\mathbf{P}, \mathbf{W}, \mathbf{C}]$ . We keep the same classification rule for these ablations, i.e., classification via Equation (10). When visual/textual prototypes are not included in the context, we use the projected features without adaptation as the matching target in Equation (9). The results are presented in Fig. 5(a). We observe that using visual prototypes or textual prototypes alone yields similar performance, and the

TABLE I  
AVERAGE AND LAST PERFORMANCE COMPARISON OF DIFFERENT METHODS

| Method             | Exemplar | Aircraft            |                 |                     |                 | CIFAR100            |                 |                     |                 | Cars                |                 |                     |                 |
|--------------------|----------|---------------------|-----------------|---------------------|-----------------|---------------------|-----------------|---------------------|-----------------|---------------------|-----------------|---------------------|-----------------|
|                    |          | B0 Inc10            |                 | B50 Inc10           |                 | B0 Inc10            |                 | B50 Inc10           |                 | B0 Inc10            |                 | B50 Inc10           |                 |
|                    |          | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ |
| Oracle             |          | 61.12               |                 |                     |                 | 82.35               |                 |                     |                 | 90.42               |                 |                     |                 |
| Finetune           | ✗        | 3.16                | 0.96            | 1.72                | 1.05            | 7.84                | 4.44            | 5.30                | 2.46            | 3.14                | 1.10            | 1.54                | 1.13            |
| Finetune LiT [95]  | ✗        | 27.74               | 14.28           | 25.10               | 13.77           | 44.66               | 14.69           | 27.69               | 7.67            | 84.12               | 72.37           | 83.08               | 78.23           |
| Finetune CoOp [33] | ✗        | 14.54               | 7.14            | 13.05               | 7.77            | 47.00               | 24.24           | 41.23               | 24.12           | 36.46               | 21.65           | 37.40               | 20.87           |
| SimpleCIL [96]     | ✗        | 59.24               | 48.09           | 53.05               | 48.09           | 84.15               | 76.63           | 80.20               | 76.63           | 92.04               | 86.85           | 88.96               | 86.85           |
| ZS-CLIP [26]       | ✗        | 26.66               | 17.22           | 21.70               | 17.22           | 81.81               | 71.38           | 76.49               | 71.38           | 82.60               | 76.37           | 78.32               | 76.37           |
| CoOp [33]          | ✓        | 44.26               | 39.87           | 41.81               | 39.18           | 83.37               | 73.36           | 78.34               | 73.04           | 89.73               | 84.91           | 87.98               | 86.60           |
| iCaRL [6]          | ✓        | 53.60               | 43.98           | 50.40               | 45.33           | 79.91               | 63.94           | 71.94               | 63.00           | 89.38               | 84.95           | 86.71               | 84.19           |
| MEMO [77]          | ✓        | 42.24               | 25.41           | 38.16               | 27.75           | 84.67               | 74.98           | 80.75               | 75.34           | 88.23               | 81.31           | 84.90               | 81.83           |
| L2P [19]           | ✓        | 55.06               | 44.88           | 47.78               | 43.37           | 76.42               | 66.21           | 72.67               | 67.88           | 83.81               | 72.44           | 79.76               | 73.47           |
| DualPrompt [18]    | ✓        | 55.95               | 46.53           | 50.93               | 46.50           | 79.07               | 70.06           | 74.81               | 70.75           | 85.30               | 74.35           | 81.32               | 75.85           |
| CODA-Prompt [97]   | ✓        | 63.13               | 52.27           | 62.05               | 54.70           | 82.91               | 74.23           | 81.33               | 75.92           | 92.11               | 89.20           | 89.45               | 87.84           |
| DAP [85]           | ✓        | 29.02               | 10.92           | 41.45               | 28.56           | 68.10               | 40.80           | 76.57               | 59.92           | 84.87               | 81.31           | 84.63               | 83.15           |
| PLOT [40]          | ✓        | 50.47               | 43.65           | 46.82               | 43.58           | 72.62               | 56.81           | 74.35               | 67.90           | 86.54               | 83.45           | 82.43               | 74.26           |
| PROOF              | ✓        | <b>64.61</b>        | <b>55.81</b>    | <b>63.59</b>        | <b>58.81</b>    | <b>86.70</b>        | <b>79.05</b>    | <b>82.92</b>        | <b>78.87</b>    | <b>93.26</b>        | <b>89.84</b>    | <b>90.53</b>        | <b>89.54</b>    |
| Method             | Exemplar | ImageNet-R          |                 |                     |                 | CUB                 |                 |                     |                 | UCF                 |                 |                     |                 |
|                    |          | B0 Inc20            |                 | B100 Inc20          |                 | B0 Inc20            |                 | B100 Inc20          |                 | B0 Inc10            |                 | B50 Inc10           |                 |
|                    |          | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ |
| Oracle             |          | 80.28               |                 |                     |                 | 79.47               |                 |                     |                 | 93.63               |                 |                     |                 |
| Finetune           | ✗        | 1.37                | 0.43            | 1.01                | 0.88            | 2.06                | 0.64            | 0.56                | 0.47            | 4.51                | 1.59            | 1.21                | 0.80            |
| Finetune LiT [95]  | ✗        | 64.88               | 30.42           | 57.75               | 29.77           | 58.15               | 35.28           | 51.95               | 35.96           | 79.25               | 64.84           | 81.79               | 65.40           |
| Finetune CoOp [33] | ✗        | 60.73               | 37.52           | 54.20               | 39.77           | 27.61               | 8.57            | 24.03               | 10.14           | 47.85               | 33.46           | 42.02               | 24.74           |
| SimpleCIL [96]     | ✗        | 81.06               | 74.48           | 76.84               | 74.48           | 83.81               | 77.52           | 79.75               | 77.52           | 90.44               | 85.68           | 88.12               | 85.68           |
| ZS-CLIP [26]       | ✗        | 83.37               | 77.17           | 79.57               | 77.17           | 74.38               | 63.06           | 67.96               | 63.06           | 75.50               | 67.64           | 71.44               | 67.64           |
| CoOp [33]          | ✓        | 82.40               | 76.20           | 79.76               | 77.13           | 77.34               | 68.70           | 74.09               | 67.47           | 90.13               | 86.24           | 88.36               | 85.71           |
| iCaRL [6]          | ✓        | 72.22               | 54.38           | 68.67               | 60.15           | 82.04               | 74.74           | 78.57               | 75.07           | 89.47               | 84.34           | 88.51               | 84.11           |
| MEMO [77]          | ✓        | 80.00               | 74.07           | 76.72               | 73.95           | 77.32               | 65.69           | 72.88               | 66.41           | 84.02               | 74.08           | 82.58               | 75.48           |
| L2P [19]           | ✓        | 75.73               | 67.22           | 74.15               | 71.20           | 79.23               | 68.54           | 75.85               | 71.12           | 88.71               | 83.93           | 86.51               | 83.22           |
| DualPrompt [18]    | ✓        | 78.47               | 70.82           | 72.98               | 69.18           | 83.21               | 74.94           | 78.06               | 74.27           | 89.48               | 85.41           | 86.96               | 84.65           |
| CODA-Prompt [97]   | ✓        | 80.91               | 74.20           | 79.32               | 74.73           | 82.91               | 73.20           | 79.81               | 74.73           | 92.51               | 89.74           | 92.73               | 90.28           |
| DAP [85]           | ✓        | 78.00               | 72.73           | 77.23               | 74.37           | 76.48               | 73.07           | 75.39               | 74.09           | 87.63               | 81.81           | 87.64               | 85.68           |
| PLOT [40]          | ✓        | 72.67               | 66.52           | 70.45               | 68.24           | 80.46               | 71.34           | 78.35               | 72.03           | 83.54               | 73.78           | 87.09               | 82.91           |
| PROOF              | ✓        | <b>85.34</b>        | <b>80.10</b>    | <b>82.32</b>        | <b>80.30</b>    | <b>84.93</b>        | <b>79.43</b>    | <b>81.67</b>        | <b>79.18</b>    | <b>94.34</b>        | <b>90.60</b>    | <b>93.56</b>        | <b>91.32</b>    |
| Method             | Exemplar | SUN                 |                 |                     |                 | Food                |                 |                     |                 | ObjectNet           |                 |                     |                 |
|                    |          | B0 Inc30            |                 | B150 Inc30          |                 | B0 Inc10            |                 | B50 Inc10           |                 | B0 Inc20            |                 | B100 Inc20          |                 |
|                    |          | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ |
| Oracle             |          | 81.74               |                 |                     |                 | 85.83               |                 |                     |                 | 45.46               |                 |                     |                 |
| Finetune           | ✗        | 4.51                | 1.59            | 0.78                | 0.72            | 3.49                | 1.71            | 2.14                | 1.52            | 1.34                | 0.47            | 0.69                | 0.54            |
| Finetune LiT [95]  | ✗        | 79.25               | 64.84           | 38.23               | 20.00           | 40.62               | 12.96           | 29.74               | 12.05           | 43.27               | 17.46           | 32.85               | 17.17           |
| Finetune CoOp [33] | ✗        | 45.93               | 23.11           | 39.33               | 24.89           | 36.01               | 14.18           | 33.13               | 18.67           | 21.24               | 6.29            | 16.21               | 6.82            |
| SimpleCIL [96]     | ✗        | 82.13               | 75.58           | 78.62               | 75.58           | 87.89               | 81.65           | 84.73               | 81.65           | 52.06               | 40.13           | 45.11               | 40.13           |
| ZS-CLIP [26]       | ✗        | 79.42               | 72.11           | 74.95               | 72.11           | 87.86               | 81.92           | 84.75               | 81.92           | 38.43               | 26.43           | 31.12               | 26.43           |
| CoOp [33]          | ✓        | 80.46               | 73.44           | 77.68               | 73.06           | 85.38               | 76.15           | 81.74               | 76.35           | 46.16               | 33.81           | 40.40               | 34.47           |
| iCaRL [6]          | ✓        | 78.56               | 67.30           | 74.74               | 69.07           | 84.12               | 71.68           | 78.86               | 70.64           | 45.28               | 26.97           | 37.22               | 26.15           |
| MEMO [77]          | ✓        | 81.48               | 73.45           | 78.00               | 73.87           | 89.18               | 82.85           | 86.50               | 83.08           | 46.98               | 33.37           | 41.62               | 34.67           |
| L2P [19]           | ✓        | 79.83               | 72.14           | 76.16               | 72.32           | 84.48               | 75.22           | 85.04               | 80.56           | 46.18               | 34.00           | 43.90               | 39.57           |
| DualPrompt [18]    | ✓        | 80.14               | 73.06           | 77.25               | 73.82           | 87.12               | 81.27           | 85.37               | 82.36           | 53.13               | 40.59           | 45.84               | 40.37           |
| CODA-Prompt [97]   | ✓        | 80.91               | 73.23           | 78.38               | 69.45           | 80.82               | 72.81           | 79.35               | 73.46           | 53.38               | 41.12           | 47.86               | 42.35           |
| DAP [85]           | ✓        | 79.81               | 73.89           | 79.61               | 74.78           | 78.49               | 71.37           | 81.68               | 78.38           | 40.10               | 37.09           | 42.47               | 32.95           |
| PLOT [40]          | ✓        | 74.91               | 62.21           | 75.34               | 68.59           | 82.37               | 74.34           | 78.39               | 72.49           | 45.34               | 34.85           | 41.85               | 33.38           |
| PROOF              | ✓        | <b>83.57</b>        | <b>77.28</b>    | <b>80.70</b>        | <b>77.49</b>    | <b>90.04</b>        | <b>84.73</b>    | <b>87.52</b>        | <b>84.74</b>    | <b>55.28</b>        | <b>44.36</b>    | <b>49.64</b>        | <b>43.65</b>    |

The first and second columns represent the methods with and without exemplars. The performance of L2P, DualPrompt, CODA-Prompt, PLOT, and DAP are reproduced with the source code with exemplars using CLIP's visual branch. The best performance is shown in bold. All methods are initialized with the same pre-trained CLIP for a fair comparison. All exemplar-related methods utilize the same number of exemplars for a fair comparison.

impact of adjustment is marginal. However, when both visual and textual prototypes are jointly utilized as context information, the model can learn from cross-modality and achieve better performance. Lastly, the introduction of context prompts into the context further enhances the performance of PROOF, resulting in the best performance among all variations.

5) *Variation of Projection Types*: Apart from simple linear layers, there are other methods to implement the projection layers, such as layer-wise rescale (SSF) [25] and Adapter [110]. SSF learns a  $d$ -dimensional rescale parameter to project the features, while Adapter learns both the down-projection and up-projection for feature mapping. We explore the performance



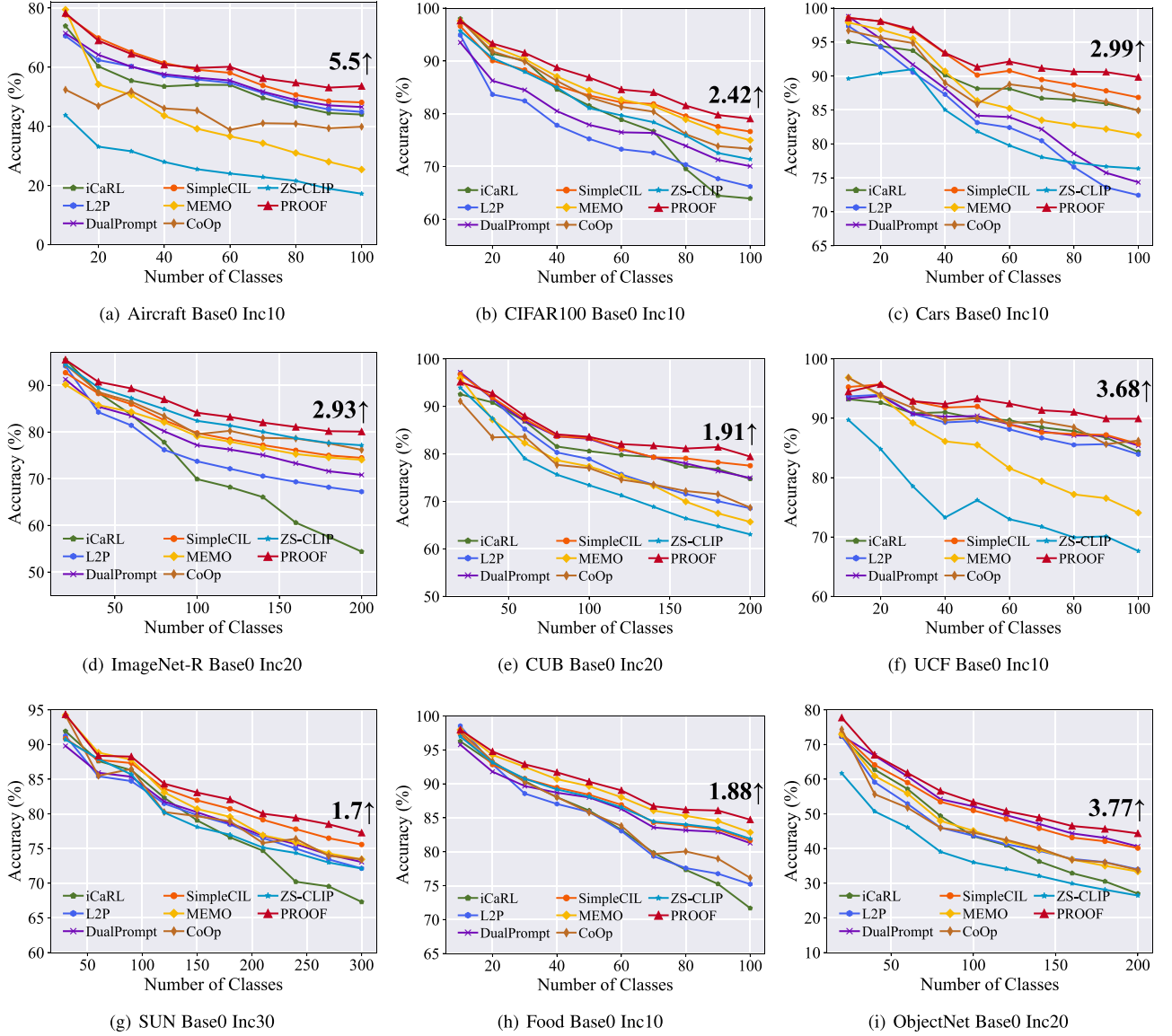


Fig. 2. Incremental performance of different methods. We report the performance gap after the last incremental stage of PROOF and the runner-up method at the end of the line. All methods are based on the same backbone/weight.

of these projection methods on CIFAR100 B0 Inc10 and present the results in Fig. 5(b). The figure clearly demonstrates that using a single linear layer as the projection layer achieves the best performance among all methods, indicating its superiority. Furthermore, this result suggests that a simple linear mapping can effectively bridge the gap between visual and textual domains.

6) *Parameter Analysis*: The additional parameters in PROOF come from three sources: the projections, the fusion module, and the visual prototypes. The projection layers are implemented with a single linear layer, each containing  $d \times d$  parameters, where  $d$  is the embedding dimension. The cross-modal fusion is implemented with a single-head self-attention mechanism, and the number of parameters is determined by the weight matrices  $W_Q$ ,  $W_K$ , and  $W_V$ , each containing  $d \times d$  parameters. The visual prototypes require saving  $B \times d$  features, where  $B$  is the number of all classes. The total number of extra parameters

is  $(2b + 3) \times d^2 + B \times d$ . Hence, these extra parameters are negligible compared to the large backbone of the pre-trained CLIP model, which has approximately 150 million parameters.

*Inference Time Merging*: As defined in Equation (6), the projected embeddings are defined as the summation of all projections. Since these projections are linear layers, we can utilize the associative law of multiplication to merge these projections:

$$P_i(\mathbf{z}) = \sum_{m=1}^b P_i^m(\mathbf{z}) = \sum_{m=1}^b P_i^m(\mathbf{z}) = \hat{P}_i(\mathbf{z}). \quad (11)$$

Eq. 11 indicates that we can merge all the projections  $(P_i^1, P_i^2, \dots, P_i^b)$  into a single one  $(\hat{P}_i)$  using the summation of the weights. Note that  $\hat{P}_i$  has the same dimension as the single projection, which means we can alleviate the storage burden of  $b$  projections into a single one. This helps us to

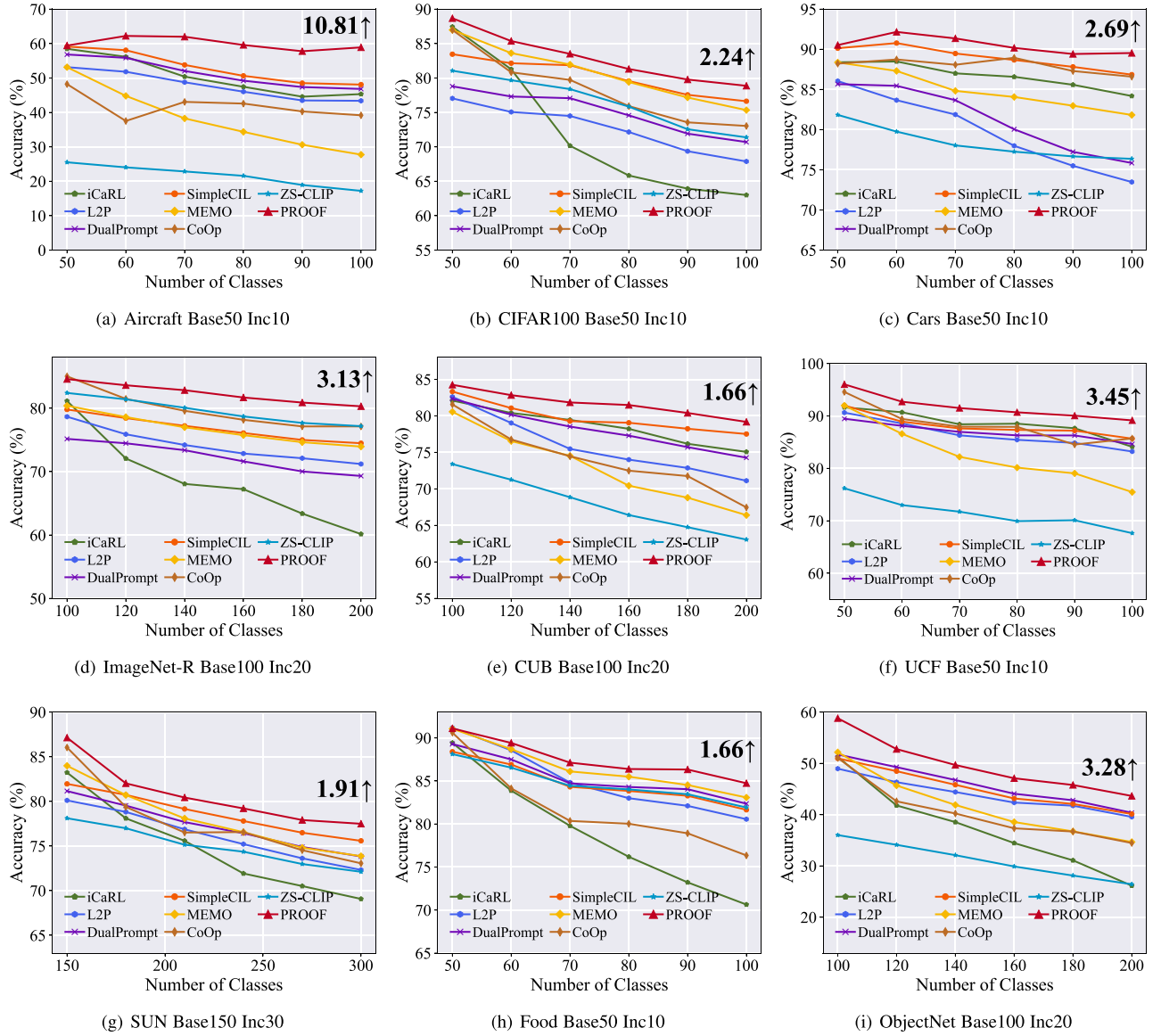


Fig. 3. Incremental performance of different methods with large base classes. We report the performance gap after the last incremental stage of PROOF and the runner-up method at the end of the line. All methods are based on the same backbone/weight.

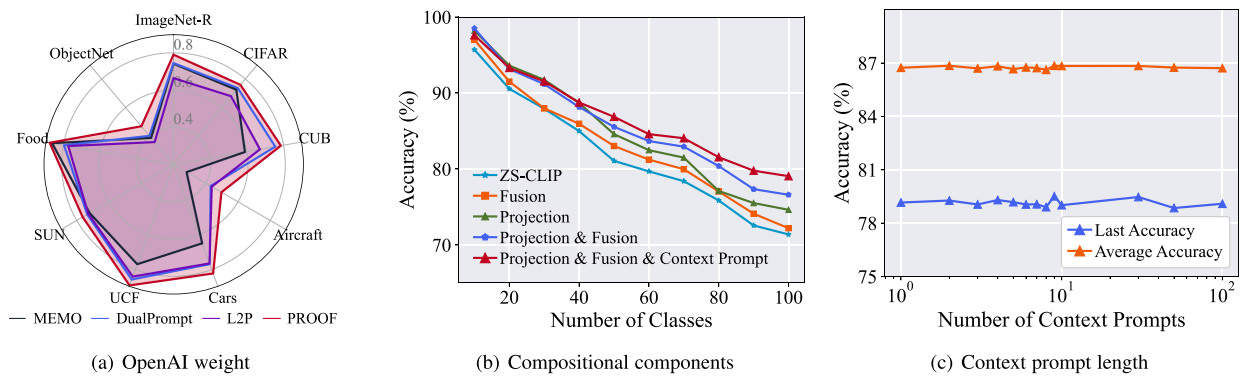


Fig. 4. Ablation study. Left: experiments on nine benchmarks with OpenAI weights. Middle: ablation study on compositional components in PROOF. Every part improves the performance of CIL. Right:  $\mathcal{A}_B$  and  $\bar{\mathcal{A}}$  with change of context prompts. The performance is robust to the change of context prompt length.

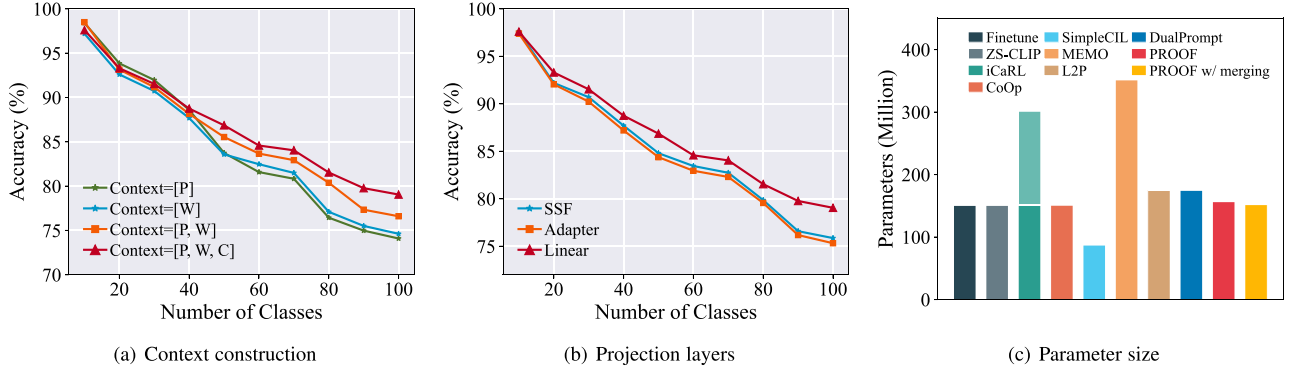


Fig. 5. Left: Variations of context information. The choice of using visual prototypes, textual prototypes, and context prompts as the context information achieves the best performance. Middle: Variations of projection layers. The choice of using a single linear layer as the projection layer achieves the best performance. Right: Number of parameters in different methods. The shaded area represents the parameters used during training but dropped during inference.

TABLE II  
EXPERIMENTS ON BY VARYING THE NUMBER OF CLASSES PER TASK

| Method           | Exemplar | CIFAR100            |                 | ImageNet-R          |                 |
|------------------|----------|---------------------|-----------------|---------------------|-----------------|
|                  |          | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ | $\bar{\mathcal{A}}$ | $\mathcal{A}_B$ |
| DualPrompt [18]  | ✓        | 82.42               | 74.09           | 79.41               | 71.68           |
| PLOT [40]        | ✓        | 77.90               | 67.66           | 70.85               | 58.63           |
| CODA-Prompt [20] | ✓        | 82.86               | 75.69           | 80.23               | 72.34           |
| DAP [85]         | ✓        | 81.12               | 73.56           | 74.38               | 73.68           |
| <b>PROOF</b>     | ✓        | <b>86.67</b>        | <b>79.75</b>    | <b>83.48</b>        | <b>78.37</b>    |

All methods are implemented with the same CLIP weight and the same number of exemplars.

decrease the extra parameters from  $(2b + 3) \times d^2 + B \times d$  to  $5 \times d^2 + B \times d$ . Since  $B$  denotes the total number of classes (which ranges from 100 to 300 in current CIL benchmarks), the second term is much smaller than the first term, and the total memory budget is limited by merging all the projections into a single one.

To provide a clear comparison of the parameter numbers for each method, we present the details in Fig. 5(c) using CIFAR100 B0 Inc10 as an example. The figure illustrates that PROOF has a similar parameter scale to other finetune-based methods while achieving significantly stronger performance. SimpleCIL, which only utilizes the vision branch, requires fewer parameters for the textual branch but lacks the zero-shot capability. L2P and DualPrompt also only require the vision branch but need an additional encoder to identify the appropriate prompt, resulting in a higher parameter count than PROOF. Additionally, PROOF with projection merging further restricts the number of parameters to be similar to a zero-shot CLIP.

7) *Different Number of Classes Per Task*: In the real world, the incremental model may face a different number of classes per task, which requires the model to tackle different numbers of classes robustly. We experiment with CIFAR100 and ImageNet-R by randomly sampling the number of classes per task, resulting in the task sequence of  $\{12, 13, 10, 14, 5, 13, 13, 14, 6\}$  for CIFAR100 and  $\{14, 25, 12, 17, 11, 27, 12, 12, 13, 21, 10, 16, 10\}$  for ImageNet-R. The performance in Table II shows that PROOF still shows substantial improvements in this real-world scenario.

#### D. Extending PROOF to Other VLMs and Other Applications

In previous sections, we use CLIP as the VLM due to its popularity and representativeness. However, the field of VLM is rapidly advancing, and various models are available. In this section, we extend PROOF to another widely used VLM, i.e., BEiT-3 [111], focusing on the cross-modal retrieval task. BEiT-3 is a popular VLM that demonstrates promising performance across multiple vision-language tasks. When finetuning BEiT-3 for cross-modal retrieval, it functions as a *dual encoder*, similar to CLIP, featuring a dual-branch structure. As the retrieval task differs from classification, we adopt a degradation of PROOF by solely employing the projection expansion strategy without implementing cross-modal fusion.

For evaluation, we employ the Flickr30K dataset [112] to assess the performance of incremental cross-modal retrieval. Flickr30K comprises 31,783 images collected from the Flickr image-sharing platform, encompassing diverse themes such as daily life, travel, people, food, and scenes. Each image in the dataset is accompanied by *five* manually annotated textual descriptions, which provide descriptive information capturing the main content and context of the images. To formulate an incremental data stream, we utilize keyword matching to identify images containing different actions (e.g., walk, stand, run, ride, play). Then, we split the training instances into five subsets based on these specific actions. To create a balanced testing set, we maintain a 5:1 training-to-testing ratio for splitting the training and testing pairs.

We employ standard cross-modal retrieval measures for evaluation, namely  $R@1$ ,  $R@5$ , and  $R@10$ . The retrieval is conducted in two directions: image  $\rightarrow$  text and text  $\rightarrow$  image. Similarly to the CIL evaluation, we report the last recall  $R_B@1$  and the average recall  $\bar{R}@1$  across incremental stages. To provide a comparative analysis, we compare PROOF against typical finetuning as the baseline and modify MEMO [77] and DER [75] for comparison. These methods represent state-of-the-art CIL approaches that can be adapted with minor modifications to the current task. However, methods such as L2P and DualPrompt are unsuitable for cross-modal retrieval tasks as they do not focus on cross-modal matching.



TABLE III  
AVERAGE AND LAST PERFORMANCE OF DIFFERENT METHODS ON CONTINUAL  
CROSS-MODAL RETRIEVAL TASKS

| Method       | Image $\rightarrow$ Text |              |              |              |              |              |
|--------------|--------------------------|--------------|--------------|--------------|--------------|--------------|
|              | $R_B@1$                  | $\bar{R}@1$  | $R_B@5$      | $\bar{R}@5$  | $R_B@10$     | $\bar{R}@10$ |
| Finetune     | 48.79                    | 62.89        | 76.38        | 85.04        | 85.68        | 91.84        |
| DER [75]     | 78.37                    | 84.48        | 96.34        | 98.23        | 99.06        | 99.59        |
| MEMO [77]    | 83.18                    | 87.79        | 96.57        | 98.27        | 99.16        | 99.66        |
| <b>PROOF</b> | <b>85.68</b>             | <b>89.43</b> | <b>97.07</b> | <b>98.68</b> | <b>99.79</b> | <b>99.86</b> |
| Method       | Text $\rightarrow$ Image |              |              |              |              |              |
|              | $R_B@1$                  | $\bar{R}@1$  | $R_B@5$      | $\bar{R}@5$  | $R_B@10$     | $\bar{R}@10$ |
| Finetune     | 37.35                    | 51.33        | 67.38        | 77.77        | 77.95        | 85.55        |
| DER [75]     | 66.71                    | 74.18        | 89.63        | 93.00        | 94.84        | 96.69        |
| MEMO [77]    | 69.53                    | 76.35        | 91.89        | 94.44        | 96.09        | 97.32        |
| <b>PROOF</b> | <b>72.10</b>             | <b>78.01</b> | <b>93.10</b> | <b>95.27</b> | <b>96.92</b> | <b>97.90</b> |

The first row stands for the text retrieval task, and the second is the image retrieval task.

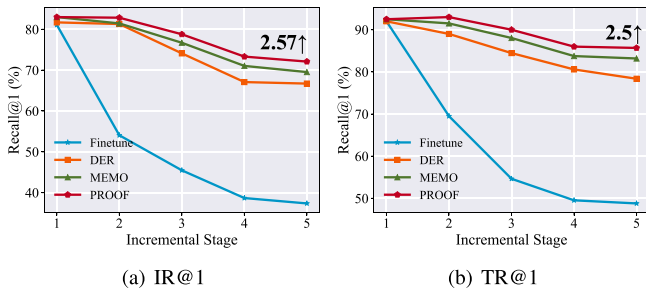


Fig. 6. Incremental performance of each method. IR means the recall of image retrieval, and TR denotes the recall of text retrieval. PROOF consistently outperforms other compared methods with a substantial margin on the continual cross-modal retrieval task.

The experimental results are presented in Table III, and Fig. 6. As evident from these figures, finetuning the model with new concepts leads to catastrophic forgetting in cross-modal retrieval tasks. However, equipping the model with incremental learning abilities alleviates forgetting. Among all the compared methods, PROOF consistently achieves the best performance across different retrieval tasks and metrics, thereby verifying its effectiveness in mitigating forgetting in VLMs. In summary, PROOF performs competitively against other algorithms even with different VLMs and continual learning settings.

#### E. CIL With Non-Overlapping Dataset

We have verified PROOF’s performance on benchmark CIL datasets in Section V-B. However, one may argue that these benchmark datasets may have data overlapping with CLIP’s pre-training dataset. Hence, we manually collect a new dataset for TV series classification with TV series after the publication of CLIP, namely TV100, for evaluation.

**Dataset Construction:** CLIP is proposed in 2021, which is trained with image-text pairs (before the year 2021) collected from the Internet. Hence, if we can collect a new dataset after 2021, we can tell that *CLIP does not know the new knowledge*. To achieve this goal, we select a field with new classes emerging every day, i.e., the TV series. Specifically, we manually search for TV series from IMDB and collect the items released after

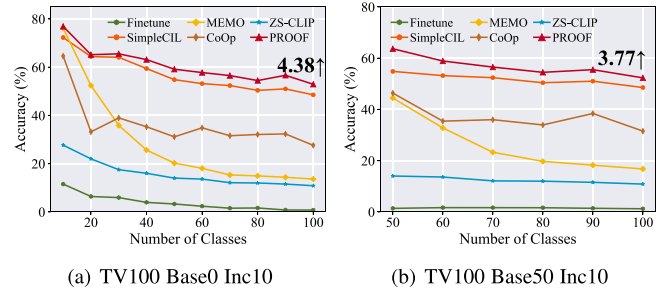


Fig. 7. Experiments on TV100, a non-overlapping dataset containing images of TV series after 2021. PROOF outperforms other compared methods by a substantial margin.

2021.<sup>1</sup> Afterward, we download the related images on Google by searching the keyword “[NAME] TV Series,” where [NAME] is the name of the TV series. The downloaded images are then processed manually to delete repeated and meaningless ones. Hence, we can get a large dataset that contains around 800 classes.

However, some of these classes may not be “new” for a pre-trained CLIP, e.g., “The Kardashians” was released in 2022 while it is not a new concept for CLIP because the Kardashian–Jenner family has been popular in America since the last century. A similar phenomenon also occurs in “The Snoopy Show” (Snoopy is a famous cartoon character) and “The Cuphead Show” (Cuphead is a video game released in 2017). Hence, we need to select some challenging classes that CLIP does not know from the TV series pool. Correspondingly, we use a pre-trained CLIP to rank the difficulty of these classes by measuring the zero-shot accuracy of each image and the text “a photo of the TV series [CLASS].” We choose the top-100 hard classes based on the zero-shot accuracy and construct the TV100 dataset. Surprisingly, a pre-trained CLIP only achieves around 10% accuracy on this dataset, verifying that CLIP does not master these classes. Besides, since the dataset is collected after the publication of CLIP, there is no class overlapping between pre-trained CLIP and TV100.

Correspondingly, we conduct experiments on this new dataset. With the other settings the same as the main paper, we select two dataset splits (i.e., Base0 Inc10 and Base50 Inc10) and report the results in Fig. 7. We can summarize two main conclusions from the figure. First, zero-shot CLIP performs poorly on this dataset, verifying that this dataset perfectly serves as the benchmark to evaluate the continual learning ability of pre-trained CLIP. Second, PROOF still outperforms other competitors by a substantial margin, verifying its strong performance in real-world continual learning tasks.

#### F. Exploring Zero-Shot Performance

CLIP is known to have the zero-shot (ZS) ability, i.e., even if the model has not been trained for recognizing the image, it can

<sup>1</sup>For those series with multiple seasons, we directly drop them since CLIP may have seen a former season, e.g., Stranger Things Season 4 is released in 2022, while Stranger Things Seasons 1, 2, and 3 are released before 2021.

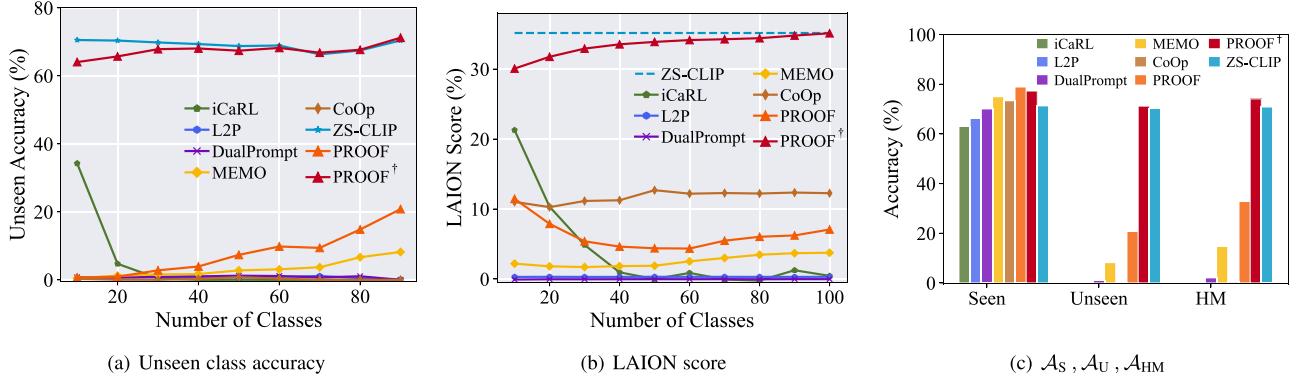


Fig. 8. Experiment on zero-shot performance. **Left:** accuracy on unseen classes during incremental learning. **Middle:** LAION score during incremental learning. **Right:** accuracy of seen, unseen, and harmonic mean (HM) at the last incremental stage. PROOF<sup>†</sup> strikes a balance between adaptivity and the ZS performance.

still predict the possibility of an image  $\mathbf{x}$  belonging to the class  $y$  by matching the cosine similarity via Equation (2). The strong generalizability of CLIP makes it a popular model in computer vision. However, in CIL, the model is *continuously* updated with the downstream task, which weakens the generalizability and harms the ZS performance [90] on subsequent tasks. In this section, we explore the zero-shot performance degradation of CLIP and propose a variation of PROOF to maintain the zero-shot performance.

**Evaluation protocol for ZS performance:** Current CIL methods focus on evaluating ‘seen’ classes, i.e., evaluating  $\mathcal{Y}_b = Y_1 \cup \dots \cup Y_b$  after learning task  $b$ . However, since CLIP exhibits ZS performance, we can also assess the performance on ‘unseen’ classes  $\mathcal{Y}_u = Y_{b+1} \cup \dots \cup Y_B$  to investigate the ZS performance. Correspondingly, we can obtain the performance metrics  $\mathcal{A}_S$  (seen classes),  $\mathcal{A}_U$  (unseen classes), and  $\mathcal{A}_{HM}$  (harmonic mean of  $\mathcal{A}_S$  and  $\mathcal{A}_U$ ) after each task. Additionally, based on the LAION-400M [113] pre-trained CLIP, we also utilize a subset of 10,000 image-text pairs from LAION-400M, and calculate the matching score of them, i.e., cosine similarity of image-text embeddings. We denote the average matching score as *LAION score*, which indicates the matching degree of the adapted model on the *upstream* tasks. Given the relationship between generalizability and the upstream task, the LAION score serves as an effective measure of ZS performance.

**Results:** We compare the aforementioned measures on CIFAR100 B0 Inc10. Apart from the compared methods in Section V-B, we also report a variation of PROOF, namely PROOF<sup>†</sup>. The only difference lies in the design of the projection, where PROOF<sup>†</sup> uses a *residual* format as the output:

$$P_i(\mathbf{z}) = \sum_{m=1}^b (P_i^m(\mathbf{z}) + \mathbf{z}), \quad P_t(\mathbf{w}) = \sum_{n=1}^b (P_t^n(\mathbf{w}) + \mathbf{w}).$$

To investigate the ZS performance as model updates, we show the accuracy on unseen classes  $\mathcal{A}_U$  along incremental stages in Fig. 8(a), where ZS-CLIP shows the best performance. Correspondingly, due to the incorporation of pre-trained information (i.e.,  $\mathbf{z}$  and  $\mathbf{w}$ ) into the projected features, PROOF<sup>†</sup> maintains

competitive ZS performance. It indicates that reflecting pre-trained information helps to maintain generalizability. Conversely, other methods experience a decline in ZS performance as their focus shifts to downstream tasks. We observe a similar trend in Fig. 8(b), where PROOF<sup>†</sup> achieves a LAION score similar to that of ZS-CLIP. Lastly, we report  $\mathcal{A}_S, \mathcal{A}_U, \mathcal{A}_{HM}$  in the last incremental stage in Fig. 8(c). We can infer a *trade-off* between the adaptivity on downstream tasks and the generalizability of ZS performance. Compared to PROOF, PROOF<sup>†</sup> sacrifices the adaptivity to maintain ZS performance, striking a balance between seen and unseen classes. Therefore, when ZS performance is essential, using PROOF<sup>†</sup> is the preferred choice.

## VI. CONCLUSION

Real-world learning systems necessitate the ability to continually acquire new knowledge. In this paper, we aim to equip the popular VLM with the CIL ability. Specifically, we learn the expandable projections so that visual and textual information can be aligned incrementally. This expansion technique allows for integrating new concepts without compromising previous ones. Additionally, we enforce cross-modality fusion with the self-attention mechanism, where visual and textual information are jointly adapted to produce instance-specific embeddings. Extensive experiments validate the effectiveness of our proposed PROOF in various VLMs and various continual learning scenarios. We also demonstrate that a simple variation can preserve the model’s zero-shot capability. Future work includes extending the model to exemplar-free scenarios.

## REFERENCES

- [1] M. Heitor, J. Gomes, P. Barddal, F. Enembreck, and A. Bifet, “A survey on ensemble learning for data stream classification,” *ACM Comput. Surv.*, vol. 50, no. 2, pp. 1–36, 2017.
- [2] W.-L. Chao, H.-J. Ye, D.-C. Zhan, M. Campbell, and K. Q. Weinberger, “Revisiting meta-learning as supervised learning,” 2020, *arXiv:2002.00573*.
- [3] C. Geng, S.-J. Huang, and S. Chen, “Recent advances in open set recognition: A survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 10, pp. 3614–3631, Oct. 2021.
- [4] F. Liu, Z. Zheng, Y. Shi, Y. Tong, and Y. Zhang, “A survey on federated learning: A perspective from multi-party computation,” *Front. Comput. Sci.*, vol. 18, no. 1, 2024, Art. no. 181336.

- [5] H.-J. Ye, D.-C. Zhan, Y. Jiang, and Z.-H. Zhou, "Heterogeneous few-shot model rectification with semantic mapping," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 11, pp. 3878–3891, Nov. 2021.
- [6] S. -A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5533–5542.
- [7] R. M. French, "Catastrophic forgetting in connectionist networks," *Trends Cogn. Sci.*, vol. 3, no. 4, pp. 128–135, 1999.
- [8] Z. Li and D. Hoiem, "Learning without forgetting," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 614–629.
- [9] M. De Lange et al., "A continual learning survey: Defying forgetting in classification tasks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3366–3385, Jul. 2022.
- [10] M. Masana, X. Liu, B. Twardowski, M. Menta, A. D. Bagdanov, and J. Van De Weijer, "Class-incremental learning: Survey and performance evaluation on image classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5513–5533, May 2023.
- [11] X. Han et al., "Pre-trained models: Past, present and future," *AI Open*, vol. 2, pp. 225–250, 2021.
- [12] M. Wen et al., "Large sequence models for sequential decision-making: A survey," *Front. Comput. Sci.*, vol. 17, no. 6, 2023, Art. no. 176349.
- [13] X. Yang, Y. Wu, M. Yang, H. Chen, and X. Geng, "Exploring diverse in-context configurations for image captioning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 40924–40943.
- [14] L. Wang et al., "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, 2024, Art. no. 186345.
- [15] Y. Wu et al., "Large scale incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 374–382.
- [16] S. Hou, X. Pan, C. C. Loy, Z. Wang, and D. Lin, "Lifelong learning via progressive distillation and retrospection," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 437–452.
- [17] B. Zhao, X. Xiao, G. Gan, B. Zhang, and S.-T. Xia, "Maintaining discrimination and fairness in class incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 13205–13214.
- [18] Z. Wang et al., "DualPrompt: Complementary prompting for rehearsal-free continual learning," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 631–648.
- [19] Z. Wang et al., "Learning to prompt for continual learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 139–149.
- [20] J. S. Smith et al., "CODA-Prompt: Continual decomposed attention-based prompting for rehearsal-free continual learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 11909–11919.
- [21] D.-W. Zhou, H.-L. Sun, J. Ning, H.-J. Ye, and D.-C. Zhan, "Continual learning with pre-trained models: A survey," in *Proc. Int. Joint Conf. Artif. Intell.*, 2024, pp. 8363–8371.
- [22] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [23] M. Jia et al., "Visual prompt tuning," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 709–727.
- [24] S. Chen et al., "AdaptFormer: Adapting vision transformers for scalable visual recognition," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 16664–16678.
- [25] D. Lian, D. Zhou, J. Feng, and X. Wang, "Scaling & shifting your features: A new baseline for efficient model tuning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 109–123.
- [26] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.
- [27] C. Jia et al., "Scaling up visual and vision-language representation learning with noisy text supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 4904–4916.
- [28] J. Yu, Z. Wang, V. Vasudevan, L. Yeung, M. Seyedhosseini, and Y. Wu, "CoCa: Contrastive captioners are image-text foundation models," *Trans. Mach. Learn. Res.*, 2022.
- [29] L. Yuan et al., "Florence: A new foundation model for computer vision," 2021, *arXiv:2111.11432*.
- [30] J. Li, D. Li, S. Savarese, and S. Hoi, "BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 19730–19742.
- [31] M. Tschanen, B. Mustafa, and N. Houlsby, "CLIPPO: Image-and-language understanding from pixels only," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 11006–11017.
- [32] J.-B. Alayrac et al., "Flamingo: A visual language model for few-shot learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 23716–23736.
- [33] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, "Learning to prompt for vision-language models," *Int. J. Comput. Vis.*, vol. 130, no. 9, pp. 2337–2348, 2022.
- [34] L. X. Li and P. Liang, "Prefix-Tuning: Optimizing continuous prompts for generation," in *Proc. Assoc. Comput. Linguistics*, 2021, pp. 4582–4597.
- [35] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, "Conditional prompt learning for vision-language models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16795–16804.
- [36] P. Gao et al., "Clip-Adapter: Better vision-language models with feature adapters," *Int. J. Comput. Vis.*, vol. 132, no. 2, pp. 581–595, 2024.
- [37] Y. Lu, J. Liu, Y. Zhang, Y. Liu, and X. Tian, "Prompt distribution learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 5196–5205.
- [38] T. Yu, Z. Lu, X. Jin, Z. Chen, and X. Wang, "Task residual for tuning vision-language models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 10899–10909.
- [39] R. Zhang et al., "Tip-Adapter: Training-free adaption of clip for few-shot classification," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 493–510.
- [40] G. Chen, W. Yao, X. Song, X. Li, Y. Rao, and K. Zhang, "PLOT: Prompt learning with optimal transport for vision-language models," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [41] G. Monge, "Mémoire sur la théorie des déblais et des remblais," *Histoire de l'Académie Royale des Sci. de Paris*, pp. 666–704, 1781.
- [42] Y.-L. Lee, Y.-H. Tsai, W.-C. Chiu, and C.-Y. Lee, "Multimodal prompting with missing modalities for visual recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14943–14952.
- [43] C. Mao et al., "Doubly right object recognition: A why prompt for visual rationales," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 2722–2732.
- [44] S. Menon and C. Vondrick, "Visual classification via description from large language models," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [45] D.-W. Zhou, Q.-W. Wang, Z.-H. Qi, H.-J. Ye, D.-C. Zhan, and Z. Liu, "Class-incremental learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 9851–9873, Dec. 2024.
- [46] H.-J. Ye, D.-W. Zhou, L. Hong, Z. Li, X.-S. Wei, and D.-C. Zhan, "Contextualizing meta-learning via learning to decompose," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 1, pp. 117–133, Jan. 2024.
- [47] Z. Luo, Y. Liu, B. Schiele, and Q. Sun, "Class-incremental exemplar compression for class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 11371–11380.
- [48] R. Aljundi, M. Lin, B. Goujaud, and Y. Bengio, "Gradient based sample selection for online continual learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 11816–11825.
- [49] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. S. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 532–547.
- [50] Y. Liu, Y. Su, A.-A. Liu, B. Schiele, and Q. Sun, "Mnemonics training: Multi-class incremental learning without forgetting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12242–12251.
- [51] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient lifelong learning with a-gem," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [52] A. Iscen, J. Zhang, S. Lazebnik, and C. Schmid, "Memory-efficient incremental learning through feature adaptation," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 699–715.
- [53] H. Zhao, H. Wang, Y. Fu, F. Wu, and X. Li, "Memory-efficient class-incremental learning for image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5966–5977, Oct. 2022.
- [54] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara, "Dark experience for general continual learning: A strong, simple baseline," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 15920–15930.
- [55] O. Ostapenko, M. Puscas, T. Klein, P. Jähne, and M. Nabi, "Learning to remember: A synaptic plasticity driven framework for continual learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11313–11321.
- [56] Y. Xiang, Y. Fu, P. Ji, and H. Huang, "Incremental learning using conditional adversarial networks," in *Proc. Int. Conf. Comput. Vis.*, 2019, pp. 6619–6628.
- [57] W. Sun, Q. Li, J. Zhang, D. Wang, W. Wang, and Y. L. Geng, "Exemplar-free class incremental learning via discriminative and comparable parallel one-class classifiers," *Pattern Recognit.*, vol. 140, 2023, Art. no. 109561.



- [58] Q. Jodelet, X. Liu, Y. J. Phua, and T. Murata, "Class-incremental learning using diffusion model for distillation and replay," in *Proc. Int. Conf. Comput. Vis. Workshop*, 2023, pp. 3425–3433.
- [59] R. Gao and W. Liu, "DDGR: Continual learning with deep diffusion-based generative replay," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 10744–10763.
- [60] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
- [61] A. Douillard, M. Cord, C. Ollion, T. Robert, and E. Valle, "PODNet: Pooled outputs distillation for small-tasks incremental learning," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 86–102.
- [62] S. Hou, X. Pan, C. C. Loy, Z. Wang, and D. Lin, "Learning a unified classifier incrementally via rebalancing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 831–839.
- [63] Y. Lu, M. Wang, and W. Deng, "Augmented geometric distillation for data-free incremental person reid," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 7329–7338.
- [64] J. Park, M. Kang, and B. Han, "Class-incremental learning for action recognition in videos," in *Proc. Int. Conf. Comput. Vis.*, 2021, pp. 13698–13707.
- [65] Q. Gao, C. Zhao, B. Ghanem, and J. Zhang, "R-DFCIL: Relation-guided representation learning for data-free class incremental learning," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 423–439.
- [66] X. Tao, X. Chang, X. Hong, X. Wei, and Y. Gong, "Topology-preserving class-incremental learning," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 254–270.
- [67] S. Dong, X. Hong, X. Tao, X. Chang, X. Wei, and Y. Gong, "Few-shot class-incremental learning via relation knowledge distillation," in *Proc. Conf. Assoc. Advance. Artif. Intell.*, 2021, pp. 1255–1263.
- [68] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci. USA*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [69] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory aware synapses: Learning what (not) to forget," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 139–154.
- [70] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3987–3995.
- [71] R. Aljundi, K. Kelchtermans, and T. Tuytelaars, "Task-free continual learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11246–11255.
- [72] Y. Shi et al., "Mimicking the oracle: An initial phase decorrelation approach for class incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16701–16710.
- [73] L. Yu et al., "Semantic drift compensation for class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 6980–6989.
- [74] E. Belouadah and A. Popescu, "IL2M: Class incremental learning with dual memory," in *Proc. Int. Conf. Comput. Vis.*, 2019, pp. 583–592.
- [75] S. Yan, J. Xie, and X. He, "DER: Dynamically expandable representation for class incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 3013–3022.
- [76] F.-Y. Wang, D.-W. Zhou, H.-J. Ye, and D.-C. Zhan, "FOSTER: Feature boosting and compression for class-incremental learning," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 398–414.
- [77] D.-W. Zhou, Q.-W. Wang, H.-J. Ye, and D.-C. Zhan, "A model or 603 exemplars: Towards memory-efficient class-incremental learning," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [78] J. Yoon, E. Yang, J. Lee, and S. J. Hwang, "Lifelong learning with dynamically expandable networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [79] J. Xu and Z. Zhu, "Reinforced continual learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 899–908.
- [80] F.-Y. Wang et al., "BEEF: Bi-compatible class-incremental learning via energy-based expansion and fusion," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [81] A. Douillard, A. Ramé, G. Couairon, and M. Cord, "DyTox: Transformers for continual learning with dynamic token expansion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 9275–9285.
- [82] Y. Hu, D. Cheng, D. Zhang, N. Wang, T. Liu, and X. Gao, "Task-aware orthogonal sparse network for exploring shared knowledge in continual learning," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 19153–19164.
- [83] K.-Y. Lee, Y. Zhong, and Y.-X. Wang, "Do pre-trained models benefit equally in continual learning," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2023, pp. 6474–6482.
- [84] Y. Lu et al., "Visual prompt tuning in null space for continual learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024.
- [85] D. Jung, D. Han, J. Bang, and H. Song, "Generating instance-level prompts for rehearsal-free continual learning," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 11847–11857.
- [86] Y.-M. Tang, Y.-X. Peng, and W.-S. Zheng, "When prompt-based incremental learning does not meet strong pretraining," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 1706–1716.
- [87] Q. Gao et al., "A unified continual learning framework with general parameter-efficient tuning," in *Proc. Int. Conf. Comput. Vis.*, 2023, pp. 11483–11493.
- [88] Y. Wang, Z. Ma, Z. Huang, Y. Wang, Z. Su, and X. Hong, "Isolation and impartial aggregation: A paradigm of incremental learning without interference," in *Proc. Conf. Assoc. Advance. Artif. Intell.*, 2023, pp. 10209–10217.
- [89] Y. Wang, Z. Huang, and X. Hong, "S-prompts learning with pre-trained transformers: An occam's razor for domain incremental learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 5682–5695.
- [90] M. Wortsman et al., "Robust fine-tuning of zero-shot models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 7949–7961.
- [91] V.W. Liang, Y.Y. Zhang, S. K. Yeung, and J. Y. Zou, "Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 17612–17625.
- [92] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4080–4090.
- [93] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [94] Z. Lin et al., "A structured self-attentive sentence embedding," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [95] X. Zhai et al., "LiT: Zero-shot transfer with locked-image text tuning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 18102–18112.
- [96] D.-W. Zhou, Z.-W. Cai, H.-J. Ye, D.-C. Zhan, and Z. Liu, "Revisiting class-incremental learning with pre-trained models: Generalizability and adaptivity are all you need," *Int. J. Comput. Vis.*, vol. 132, pp. 1–21, 2024.
- [97] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Master's thesis, Univ. Tront, 2009.
- [98] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD Birds-200–2011 dataset," California Institute of Technology, Tech. Rep. CNS-TR-2011-001, 2011.
- [99] A. Barbu et al., "ObjectNet: A large-scale bias-controlled dataset for pushing the limits of object recognition models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 9448–9458.
- [100] D. Hendrycks et al., "The many faces of robustness: A critical analysis of out-of-distribution generalization," in *Proc. Int. Conf. Comput. Vis.*, 2021, pp. 8340–8349.
- [101] S. Maji, E. Rahtu, J. Kannala, M. Blaschko, and A. Vedaldi, "Fine-grained visual classification of aircraft," 2013, *arXiv:1306.5151*.
- [102] J. Krause, M. Stark, J. Deng, and L. Fei-Fei, "3D object representations for fine-grained categorization," in *Proc. Int. Conf. Comput. Vis. Workshop*, 2013, pp. 554–561.
- [103] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101—mining discriminative components with random forests," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 446–461.
- [104] J. Xiao, J. Hays, K. A. Ehinger, A. Oliva, and A. Torralba, "SUN database: Large-scale scene recognition from abbey to zoo," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2010, pp. 3485–3492.
- [105] K. Soomro, A. R. Zamir, and M. Shah, "UCF101: A dataset of 101 human actions classes from videos in the wild," 2012, *arXiv:1212.0402*.
- [106] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.
- [107] C. Cherti et al., "Reproducible scaling laws for contrastive language-image learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 2818–2829.
- [108] M. Welling, "Herdling dynamical weights to learn," in *Proc. Int. Conf. Mach. Learn.*, 2009, pp. 1121–1128.
- [109] N. Mu, A. Kirillov, D. Wagner, and S. Xie, "SLIP: Self-supervision meets language-image pre-training," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 529–544.
- [110] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2790–2799.



- [111] W. Wang et al., "Image as a foreign language: BEIT pretraining for vision and vision-language tasks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 19175–19186.
- [112] B. A. Plummer, L. Wang, C. M. Cervantes, J. C. Caicedo, J. Hockenmaier, and S. Lazebnik, "Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models," in *Proc. Int. Conf. Comput. Vis.*, 2015, pp. 2641–2649.
- [113] C. Schuhmann et al., "LAION-400M: Open dataset of clip-filtered 400 million image-text pairs," 2021, *arXiv:2111.02114*.



**Da-Wei Zhou** received the PhD degree in computer science from Nanjing University, China, in 2024. Currently, he holds the position of associate researcher with the National Key Lab for Novel Software Technology, the School of Artificial Intelligence, Nanjing University. His research interests lie primarily in machine learning and computer vision. He has published extensively on top-tier conferences and journals in relevant fields, including NeurIPS, ICLR, ICML, CVPR, ECCV, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *International*

*Journal of Computer Vision*, etc. He serves as Program Committee member in leading conferences such as ICML, NeurIPS, ICLR, CVPR, etc. He was recognized as NeurIPS's top reviewer and CVPR's outstanding reviewer.



**Jingyi Ning** received the PhD degree in computer science from Nanjing University, China in 2024. She is currently an assistant researcher with the School of Computer Science at Nanjing University. Her research interests include intelligent sensing, mobile computing, and multi-modal sensing.



**Han-Jia Ye** received the PhD degree in computer science from Nanjing University, China, in 2019. He joined the School of Artificial Intelligence, Nanjing University as a faculty member in 2019 and currently holds the position of associate professor. His research focuses primarily on machine learning, with interests in representation learning, model reuse, and meta-learning. He has served as the tutorial co-chair for SDM 2023. Additionally, he participates as area chairs in conferences, such as ICML, NeurIPS, and CVPR, and others.



**Yuanhan Zhang** is currently working toward the PhD degree with MMLab@NTU, Nanyang Technological University, supervised by Prof. Ziwei Liu. His interests lie in computer vision and deep learning. In particular, He is focused on adapting foundation models—from vision to multi-modal—for real-world exploration. He has published several papers in ECCV, NeurIPS and *IEEE Transactions on Pattern Analysis and Machine Intelligence* (TPAMI). He also served as a reviewer for CVPR, ICCV, ECCV, NeurIPS, ICML, ICLR, *IEEE Transactions on Pattern Analysis and Machine Intelligence* (TPAMI), and *International Journal of*

*Computer Vision* (IJCV).



**De-Chuan Zhan** received the PhD degree in computer science, Nanjing University, China in 2010. In 2010, he became a faculty member with the Department of Computer Science and Technology, Nanjing University, China. He is currently a professor with the School of Artificial Intelligence, Nanjing University. His research interests are mainly in machine learning, data mining, and mobile intelligence. He has published more than 90 papers in leading international journal/conferences. He serves as an editorial board member of IDA and IJAPR, and serves as SPC/PC in leading conferences such as IJCAI, AAAI, ICML, NeurIPS, etc.



**Yan Wang** is currently working toward the MSc degree with the National Key Lab for Novel Software Technology, School of Artificial Intelligence, Nanjing University, China. His research interests include primarily in machine learning and data mining, including incremental learning.



**Ziwei Liu** (Member, IEEE) is currently a nanyang assistant professor with Nanyang Technological University, Singapore. His research interests include computer vision machine learning and computer graphics. He has published extensively on top-tier conferences and journals in relevant fields, including CVPR, ICCV, ECCV, NeurIPS, ICLR, ICML, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *ACM Transactions on Graphics* and *Nature - Machine Intelligence*. He is the recipient of ICCV Young Researcher Award, HKSTP Best Paper Award, CVPR Best Paper Award Candidate, ICBS Frontiers of Science Award and MIT Technology Review Innovators under 35 Asia Pacific. He serves as an area chair of CVPR, ICCV, ECCV, NeurIPS and ICLR, as well as an associate editor of *International Journal of Computer Vision*.